



**Ho Chi Minh City University of Technology**  
**Faculty of Computer Science and Engineering**

# **Chapter 2: Divide-and-conquer**

---

## **Algorithms Analysis and Design (C03031)**

Instructor: Assoc. Prof. Dr. Vo Thi Ngoc Chau

Email: [chauvtn@hcmut.edu.vn](mailto:chauvtn@hcmut.edu.vn)



# Course Learning Outcomes

---

1. Able to analyze the complexity of the algorithms (recursive or iterative) and estimate the efficiency of the algorithms.
2. Improve the ability to design algorithms in different areas *by decomposing a computing problem into advanced algorithm design strategies.*
3. Able to discuss NP-completeness.



# References

---

- [1] Cormen, T. H., Leiserson, C. E, and Rivest, R. L., *Introduction to Algorithms*, The MIT Press, 2009.
- [2] Levitin, A., *Introduction to the Design and Analysis of Algorithms*, 3<sup>rd</sup> Edition, Pearson, 2012.
- [3] Sedgewick, R., *Algorithms in C++*, Addison-Wesley, 1998.
- [4] Weiss, M.A., *Data Structures and Algorithm Analysis in C*, The Benjamin/Cummings Publishing, 1993.



# Course Outline

---

- ❑ Chapter 1. Fundamentals
- ❑ **Chapter 2. Divide-and-Conquer Strategy**
- ❑ Chapter 3. Decrease-and-Conquer Strategy
- ❑ Chapter 4. Transform-and-Conquer Strategy
- ❑ Chapter 5. Dynamic Programming and Greedy Strategies
- ❑ Chapter 6. Backtracking and Branch-and-Bound
- ❑ Chapter 7. NP-completeness
- ❑ Chapter 8. Approximation Algorithms



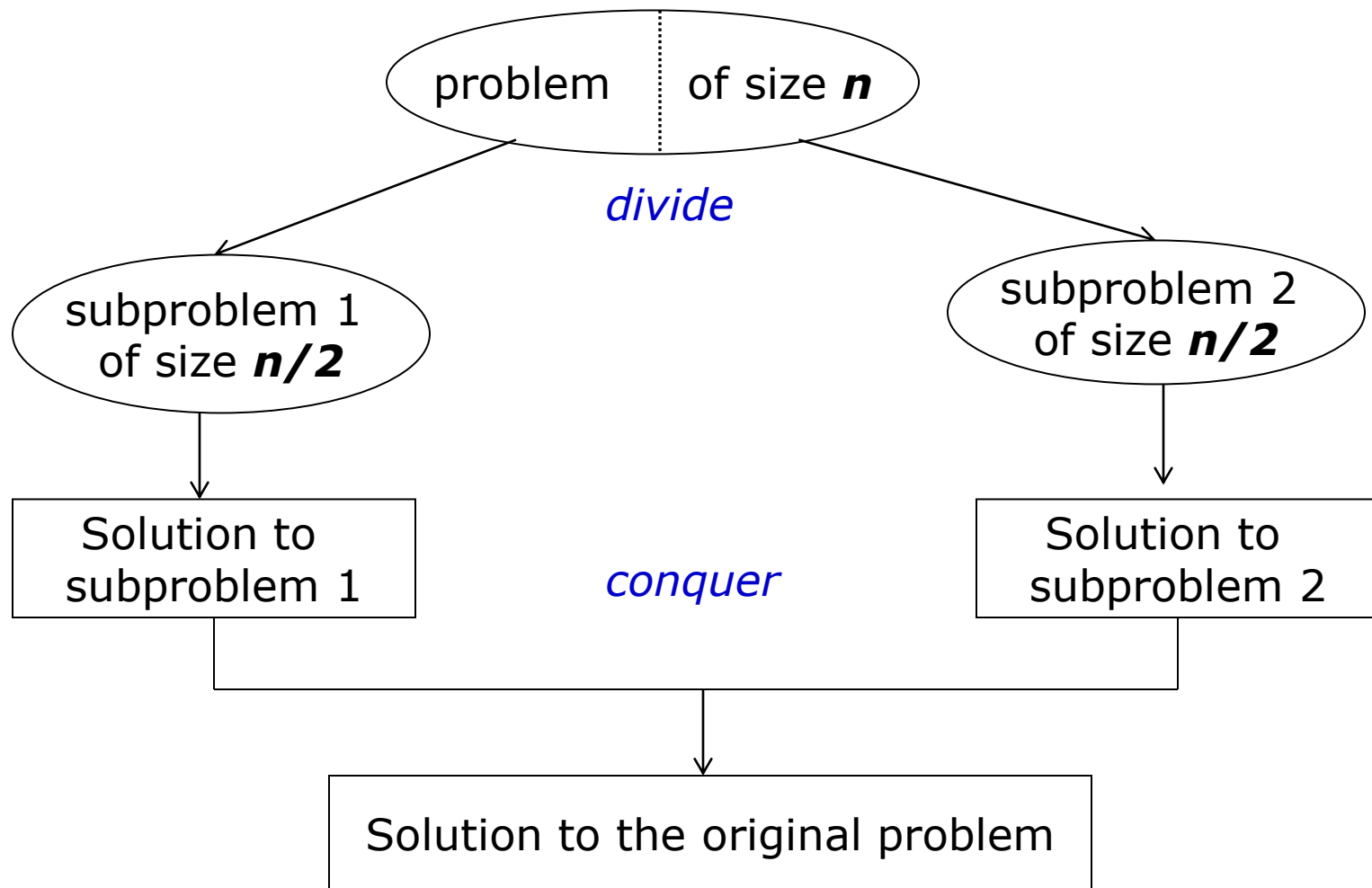
# Chapter 2.

## Divide-and-Conquer Strategy

---

- 2.1. Divide-and-conquer strategy
- 2.2. Quicksort
- 2.3. Mergesort
- 2.4. External sorting
- 2.5. Binary search tree

## 2.1. Divide-and-conquer strategy



Divide-and-conquer

Figure 4.1, [2], pp. 124.



## 2.2. Quicksort

---

- ❑ The basic algorithm of Quicksort was invented in 1960 by C. A. R. Hoare.
- ❑ Quicksort is a “divide-and-conquer” method for sorting by *partitioning* the input file into two parts, then sorting each part independently.
- ❑ ***Partitioning*** rearranges the array to make the following three conditions hold:
  - i. the element  $a[k]$  sorted in its final place in the array for some  $k$
  - ii. all the elements in  $a[left], \dots, a[k-1]$  less than or equal to  $a[k]$
  - iii. all the elements in  $a[k+1], \dots, a[right]$  greater than or equal to  $a[k]$



## 2.2. Quicksort

---

### □ 2.1. Quicksort – pivot = leftmost

8 8 9 2 1 0 0 7 6 11 4 9



0 0 1 2 4 6 7 8 8 9 9 11





## 2.2. Quicksort

### □ 2.1. Quicksort – pivot = leftmost

```
procedure quicksort (left, right:integer);  
var i: integer;  
begin  
    if right > left then  
        begin  
            k := partitioning (left, right);  
            quicksort (left, k-1);  
            quicksort (k+1, right);  
        end;  
end;
```

## 2.2. Quicksort

### □ 2.1. Quicksort – pivot = leftmost

8 8 9 2 1 0 0 7 6 11 4 9  
**8** 8 9 2 1 0 0 7 6 11 4 9

*Partitioning*

7 4 6 2 1 0 0 **8** 9 11 8 9

Unsorted left      **Sorted**      Unsorted right

*Partitioning:*

**Red:** pivot

From left to right:

**Blue:**  $j, \geq$

From right to left:

**Green:**  $k, \leq$

$j < k$ : swap ( $a[j]$ ,  $a[k]$ )

$j \geq k$ : swap ( $a[k]$ , pivot)



## 2.2. Quicksort

### □ 2.1. Quicksort – pivot = leftmost

8 8 9 2 1 0 0 7 6 11 4 9

8 8 9 2 1 0 0 7 6 11 4 9

*Partitioning:*

Red: pivot

From left to right:

Blue:  $j, \geq$

From right to left:

Green:  $k, \leq$

$j < k$ : swap ( $a[j]$ ,  $a[k]$ )

$j \geq k$ : swap ( $a[k]$ , pivot)



## 2.2. Quicksort

### □ 2.1. Quicksort – pivot = leftmost

8 8 9 2 1 0 0 7 6 11 4 9

8 8 9 2 1 0 0 7 6 11 4 9  
 $j \rightarrow$

*Partitioning:*

Red: pivot

From left to right:

Blue:  $j, \geq$

From right to left:

Green:  $k, \leq$

$j < k$ : swap ( $a[j]$ ,  $a[k]$ )

$j \geq k$ : swap ( $a[k]$ , pivot)



## 2.2. Quicksort

### □ 2.1. Quicksort – pivot = leftmost

8 8 9 2 1 0 0 7 6 11 4 9  
8 8 9 2 1 0 0 7 6 11 4 9  
 $j \rightarrow$   $\leftarrow k$

*Partitioning:*

Red: pivot

From left to right:

Blue:  $j, \geq$

From right to left:

Green:  $k, \leq$

$j < k$ : swap ( $a[j]$ ,  $a[k]$ )

$j \geq k$ : swap ( $a[k]$ , pivot)



## 2.2. Quicksort

### □ 2.1. Quicksort – pivot = leftmost

8 8 9 2 1 0 0 7 6 11 4 9

8 8 9 2 1 0 0 7 6 11 4 9

$j \rightarrow$   $\leftarrow k$

8 4 9 2 1 0 0 7 6 11 8 9

*Partitioning:*

Red: pivot

From left to right:

Blue:  $j, \geq$

From right to left:

Green:  $k, \leq$

$j < k$ : swap ( $a[j]$ ,  $a[k]$ )

$j \geq k$ : swap ( $a[k]$ , pivot)



## 2.2. Quicksort

### □ 2.1. Quicksort – pivot = leftmost

8 8 9 2 1 0 0 7 6 11 4 9

8 8 9 2 1 0 0 7 6 11 4 9

$j \rightarrow$   $\leftarrow k$

8 4 9 2 1 0 0 7 6 11 8 9

$j \rightarrow$

*Partitioning:*

Red: pivot

From left to right:

Blue:  $j, \geq$

From right to left:

Green:  $k, \leq$

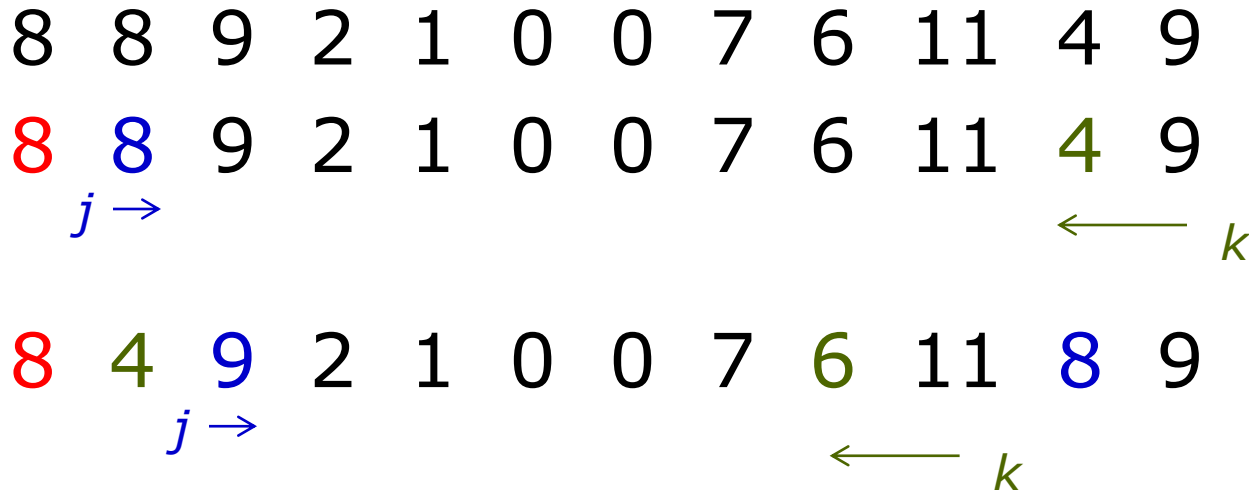
$j < k$ : swap ( $a[j]$ ,  $a[k]$ )

$j \geq k$ : swap ( $a[k]$ , pivot)



## 2.2. Quicksort

### □ 2.1. Quicksort – pivot = leftmost



*Partitioning:*

Red: pivot

From left to right:

Blue:  $j, \geq$

From right to left:

Green:  $k, \leq$

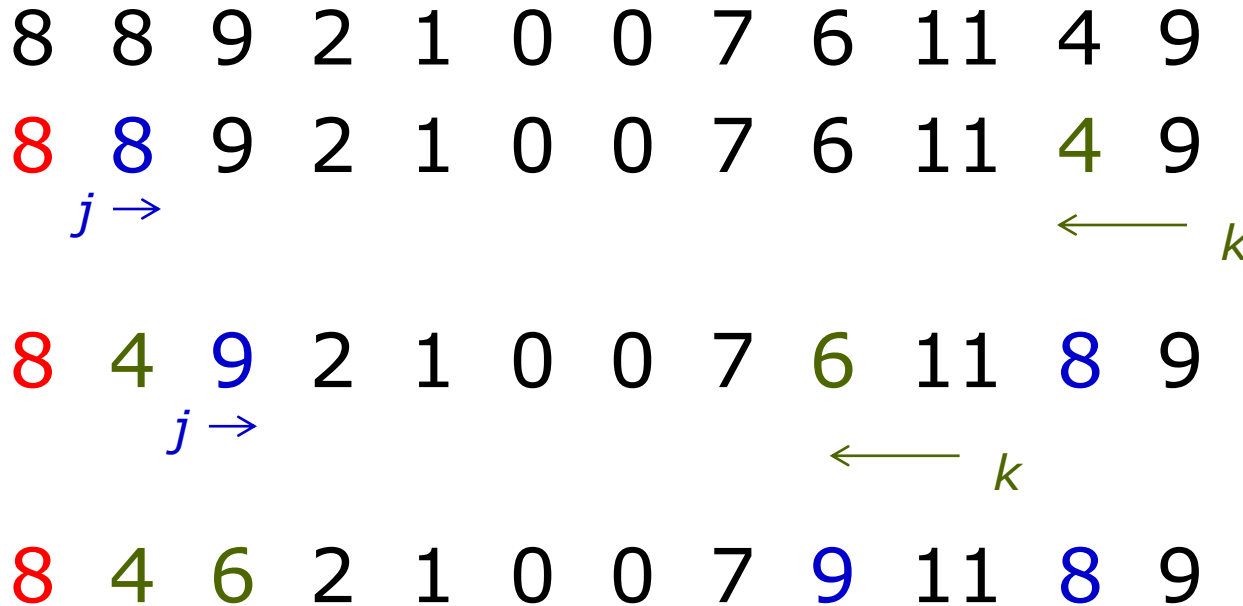
$j < k$ : swap ( $a[j]$ ,  $a[k]$ )

$j \geq k$ : swap ( $a[k]$ , pivot)



## 2.2. Quicksort

### □ 2.1. Quicksort – pivot = leftmost



*Partitioning:*

Red: pivot

From left to right:

Blue:  $j, \geq$

From right to left:

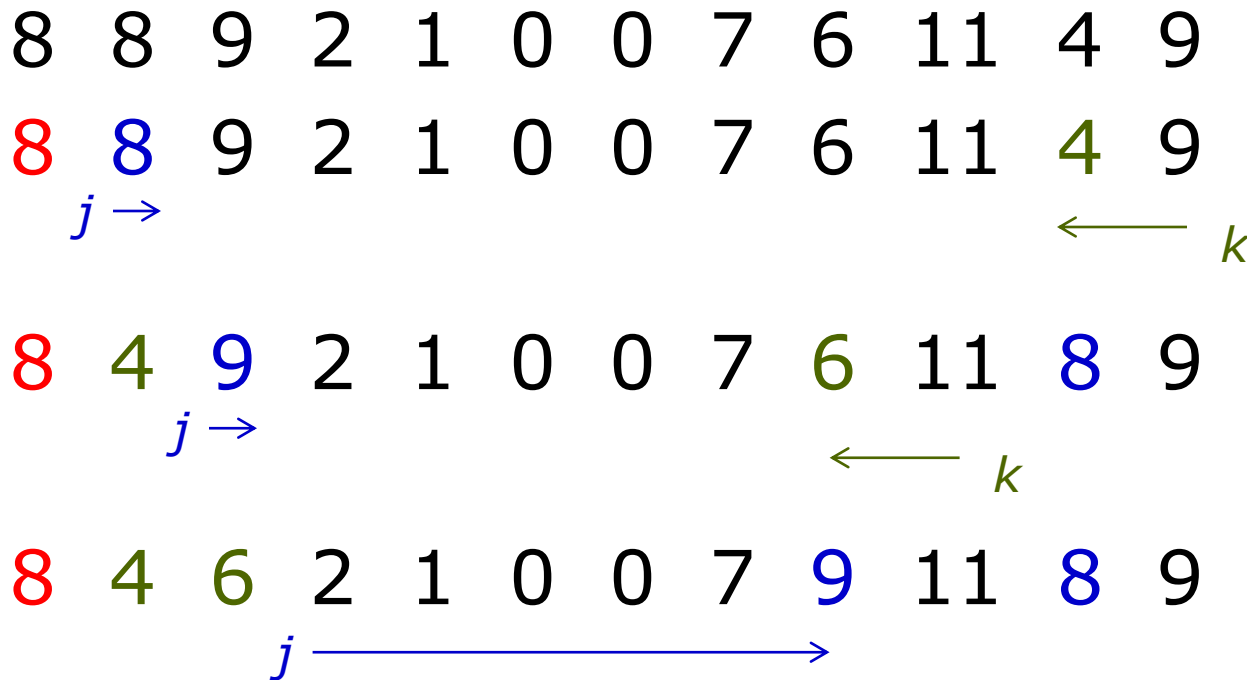
Green:  $k, \leq$

$j < k$ : swap ( $a[j]$ ,  $a[k]$ )

$j \geq k$ : swap ( $a[k]$ , pivot)

## 2.2. Quicksort

### □ 2.1. Quicksort – pivot = leftmost



*Partitioning:*

Red: pivot

From left to right:

Blue:  $j, \geq$

From right to left:

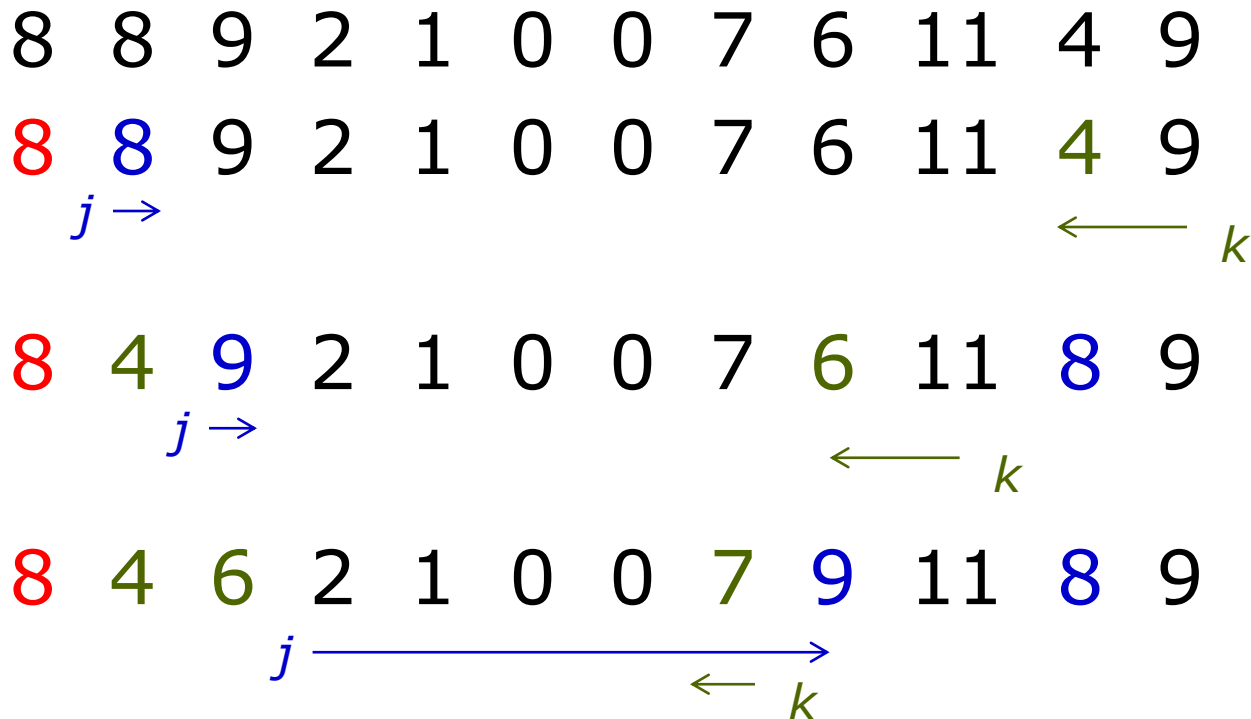
Green:  $k, \leq$

$j < k$ : swap ( $a[j]$ ,  $a[k]$ )

$j \geq k$ : swap ( $a[k]$ , pivot)

## 2.2. Quicksort

### □ 2.1. Quicksort – pivot = leftmost



*Partitioning:*

Red: pivot

From left to right:

Blue:  $j, \geq$

From right to left:

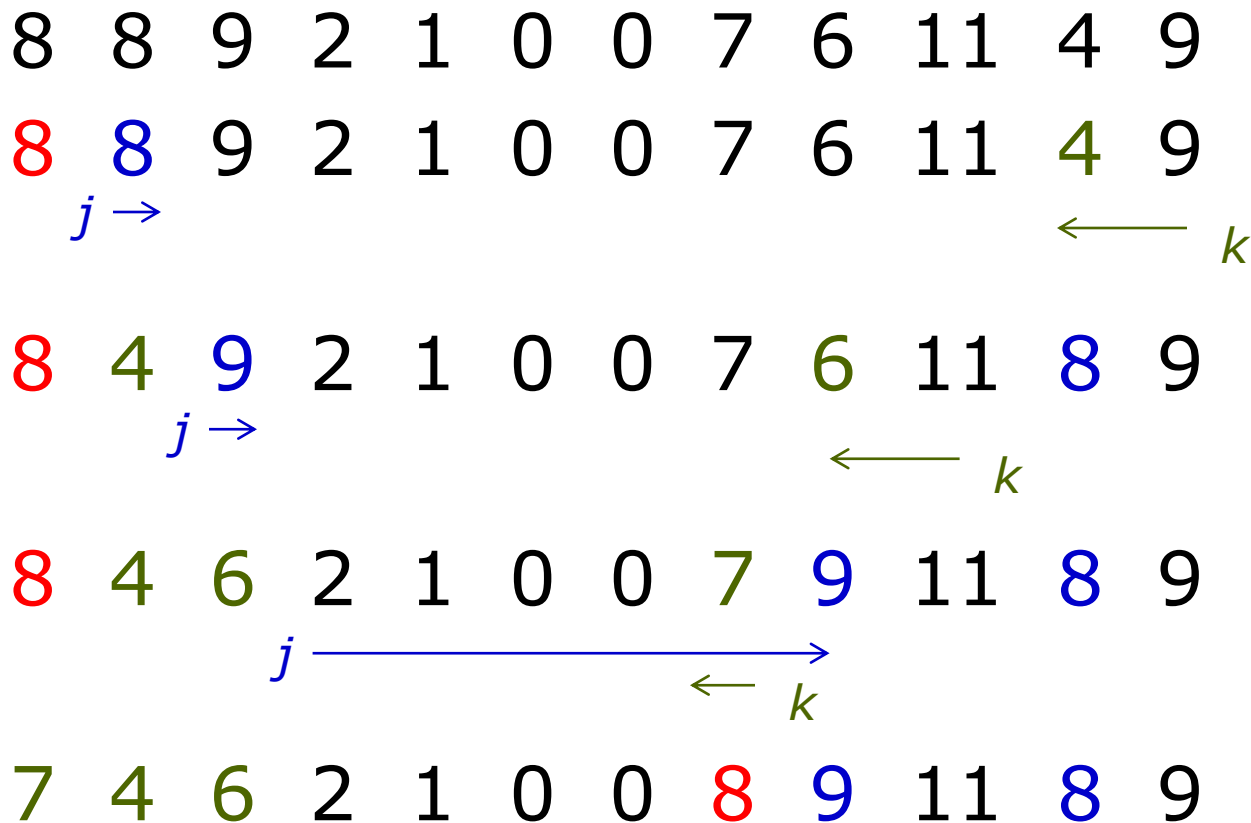
Green:  $k, \leq$

$j < k$ : swap ( $a[j]$ ,  $a[k]$ )

$j \geq k$ : swap ( $a[k]$ , pivot)

## 2.2. Quicksort

### □ 2.1. Quicksort – pivot = leftmost



*Partitioning:*

Red: pivot

From left to right:

Blue:  $j, \geq$

From right to left:

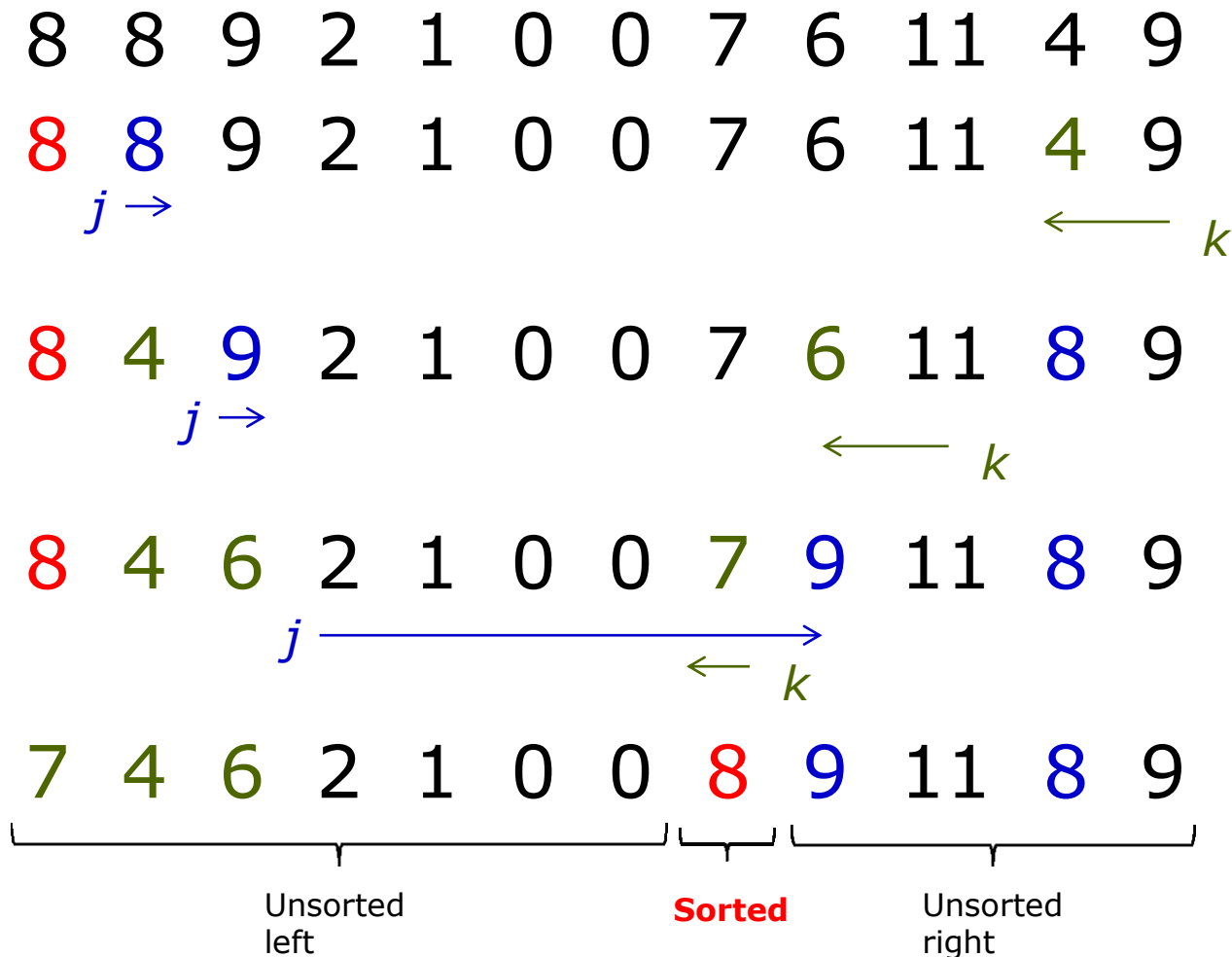
Green:  $k, \leq$

$j < k$ : swap ( $a[j]$ ,  $a[k]$ )

$j \geq k$ : swap ( $a[k]$ , pivot)

## 2.2. Quicksort

### □ 2.1. Quicksort – pivot = leftmost



*Partitioning:*

Red: pivot

From left to right:

Blue:  $j, \geq$

From right to left:

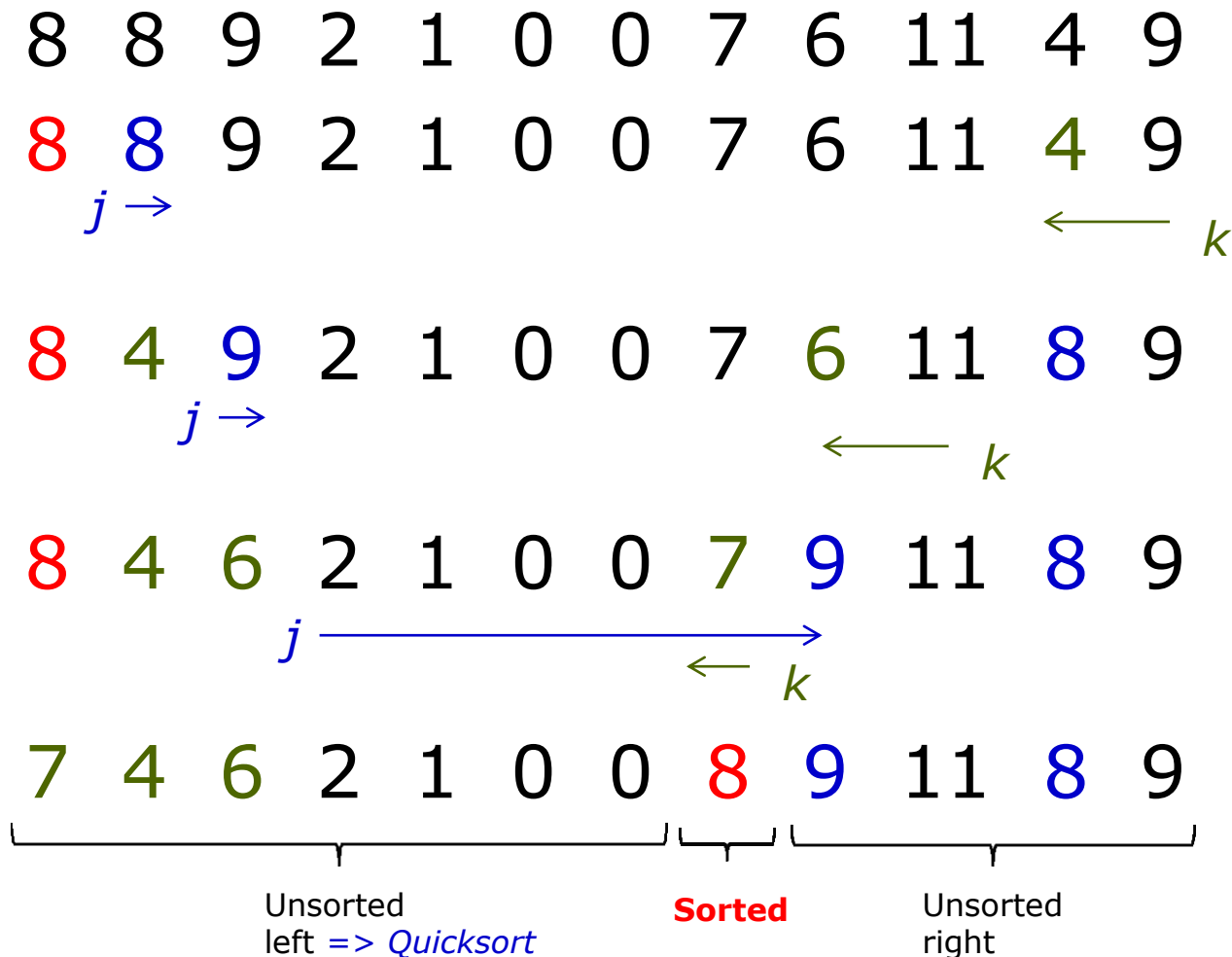
Green:  $k, \leq$

$j < k$ : swap ( $a[j]$ ,  $a[k]$ )

$j \geq k$ : swap ( $a[k]$ , pivot)

## 2.2. Quicksort

### 2.1. Quicksort – pivot = leftmost



*Partitioning:*

Red: pivot

From left to right:

Blue:  $j, \geq$

From right to left:

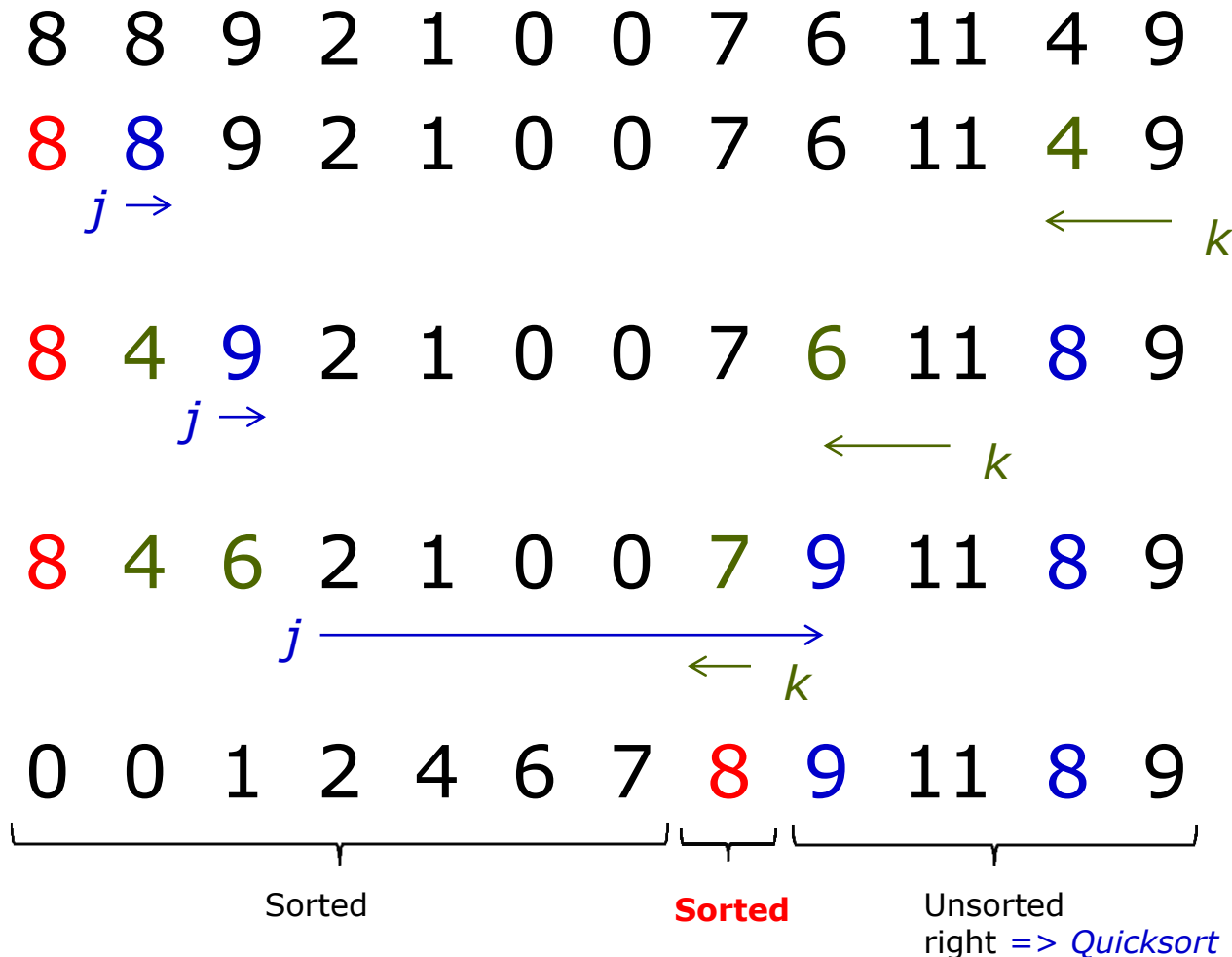
Green:  $k, \leq$

$j < k$ : swap ( $a[j]$ ,  $a[k]$ )

$j \geq k$ : swap ( $a[k]$ , pivot)

## 2.2. Quicksort

### □ 2.1. Quicksort – pivot = leftmost



*Partitioning:*

Red: pivot

From left to right:

Blue:  $j, \geq$

From right to left:

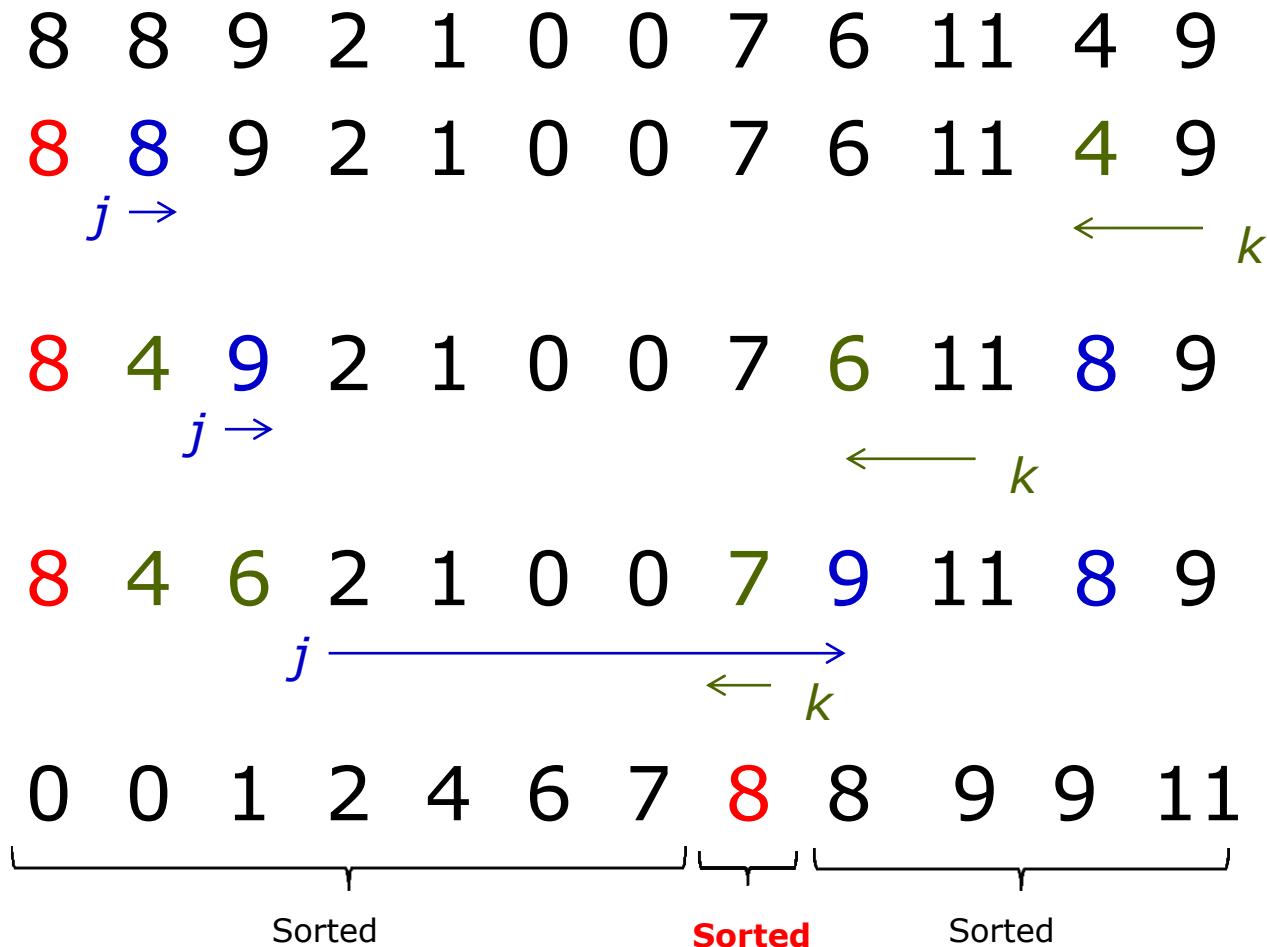
Green:  $k, \leq$

$j < k$ : swap ( $a[j]$ ,  $a[k]$ )

$j \geq k$ : swap ( $a[k]$ , pivot)

## 2.2. Quicksort

### □ 2.1. Quicksort – pivot = leftmost



*Partitioning:*

Red: pivot

From left to right:

Blue:  $j, \geq$

From right to left:

Green:  $k, \leq$

$j < k$ : swap ( $a[j]$ ,  $a[k]$ )

$j \geq k$ : swap ( $a[k]$ , pivot)



## 2.2. Quicksort

```
procedure quicksort (left, right: integer);  
var j, k: integer;  
begin  
    if right > left then  
        begin  
            j:=left; k:=right+1;  
            repeat //start partitioning  
                repeat j:=j+1 until a[j] >= a[left];  
                repeat k:=k-1 until a[k]<= a[left];  
                if j< k then swap(a[j],a[k])  
            until j>=k;  
            swap(a[left],a[k]); //finish partitioning  
            quicksort (left,k-1);  
            quicksort (k+1,right)  
        end;  
end;
```



## 2.2. Quicksort

---

- Time complexity for sorting  $n$  elements

- Abstract operation: *comparison*

- Best case:

$$C_n = n \log_2 n = O(n \log_2 n)$$

- Average case:

$$C_n = 2n \ln n = 1.38n \log_2 n = O(n \log_2 n)$$

- Worst case:

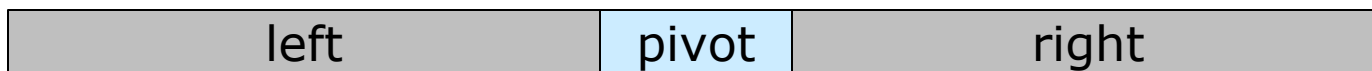
$$C_n = (n^2 + 3n - 4)/2 = O(n^2)$$



## 2.2. Quicksort

- Time complexity for sorting  $n$  elements
  - Abstract operation: *comparison*
  - Best case:

$$C_n = n \log_2 n = O(n \log_2 n)$$



Partitioning gives two halves of similar size:  $n/2$ .

$$C_1 = 0$$

$$C_n = \underbrace{2C_{n/2}}_{\substack{\text{Sort left} \\ + \text{Sort right}}} + \underbrace{(n+1)}_{\substack{\text{Partitioning} \\ \text{when } j \text{ crosses } k}}$$

or

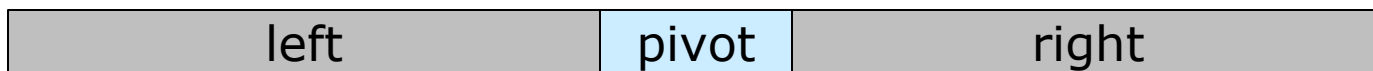
$$C_n = \underbrace{2C_{n/2}}_{\substack{\text{Sort left} \\ + \text{Sort right}}} + \underbrace{n}_{\substack{\text{Partitioning} \\ \text{when } j \text{ \& } k \text{ coincide}}}$$



## 2.2. Quicksort

- Time complexity for sorting  $n$  elements
  - Abstract operation: *comparison*
  - Best case:

$$C_n = n \log_2 n = O(n \log_2 n)$$



Partitioning gives two halves of similar size:  $n/2$ .

$$C_1 = 0$$

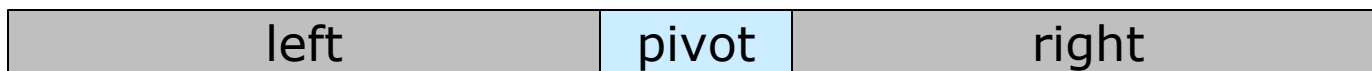
$$C_n = 2C_{n/2} + (n+1) \quad \text{or} \quad \mathbf{C_n = 2C_{n/2} + n}$$

$$C_n = n \log_2 n + n - 1 \quad \mathbf{C_n = n \log_2 n}$$

## 2.2. Quicksort

- Time complexity for sorting  $n$  elements
  - Abstract operation: *comparison*
  - Best case:

$$C_n = n \log_2 n = O(n \log_2 n)$$



Partitioning gives two halves of similar size:  $n/2$ .

$$C_1 = 0$$

$$C_n = 2C_{n/2} + (n+1) \quad \text{or} \quad \mathbf{C_n = 2C_{n/2} + n}$$

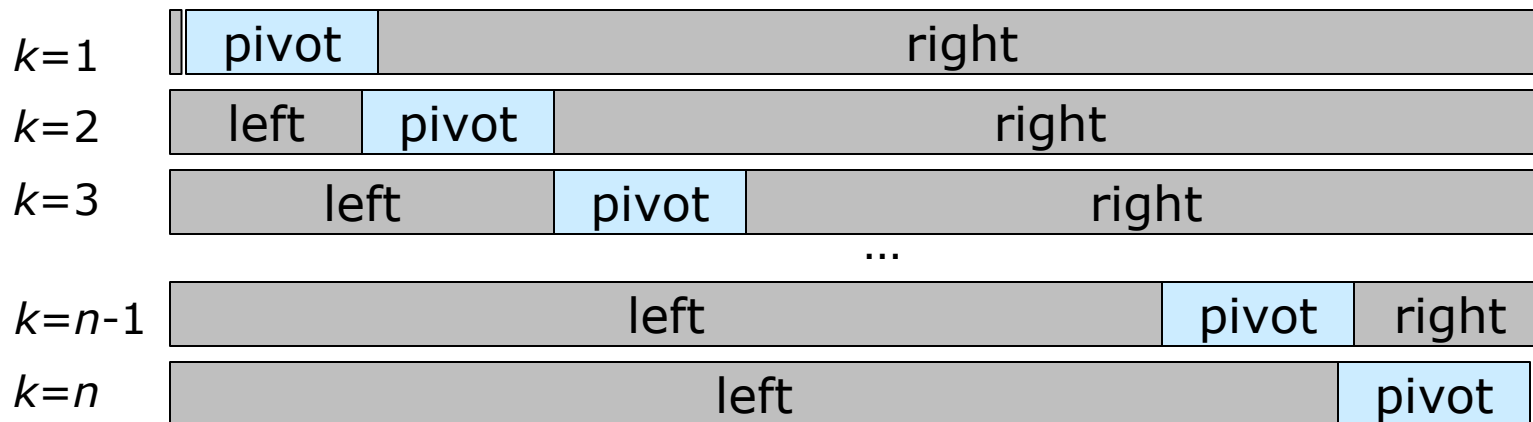
$$C_n = n \log_2 n + n - 1 \quad \mathbf{C_n = n \log_2 n}$$



## 2.2. Quicksort

- Time complexity for sorting  $n$  elements
  - Abstract operation: *comparison*
  - Average case:

$$C_n = 2n \ln n = 1.38n \log_2 n = O(n \log_2 n)$$



Partition split can happen in each position ( $1 \leq k \leq n$ ) with the same probability  $1/n$ .

$$C_0 = C_1 = 0$$

$$C_n = \frac{1}{n} \sum_{k=1}^n (C_{k-1} + C_{n-k} + (n+1)) \text{ when } n > 1$$



## 2.2. Quicksort

- Time complexity for sorting  $n$  elements
  - Abstract operation: *comparison*
  - Average case:

$$C_n = 2n \ln n = 1.38n \log_2 n = O(n \log_2 n)$$

Partition split can happen in each positions ( $1 \leq k \leq n$ ) with the same probability  $1/n$ .

$$C_0 = C_1 = 0$$

$$C_n = \frac{1}{n} \sum_{k=1}^n (C_{k-1} + C_{n-k} + (n+1)) \text{ when } n > 1$$

$$C_n = n + 1 + \frac{1}{n} \sum_{k=1}^n (C_{k-1} + C_{n-k})$$

$$\rightarrow nC_n = n(n+1) + \sum_{k=1}^n (C_{k-1} + C_{n-k})$$

$$\rightarrow (n-1)C_{n-1} = (n-1)n + \sum_{k=1}^{n-1} (C_{k-1} + C_{n-1-k})$$

$$\rightarrow nC_n - (n-1)C_{n-1} = n(n+1) - (n-1)n + C_{n-1} + C_{n-1}$$



## 2.2. Quicksort

- Time complexity for sorting  $n$  elements
  - Abstract operation: *comparison*
  - Average case:

$$C_n = 2n \ln n = 1.38n \log_2 n = O(n \log_2 n)$$

Partition split can happen in each positions ( $1 \leq k \leq n$ ) with the same probability  $1/n$ .

$$C_0 = C_1 = 0$$

$$C_n = \frac{1}{n} \sum_{k=1}^n (C_{k-1} + C_{n-k} + (n+1)) \text{ when } n > 1$$

$$C_n = n + 1 + \frac{1}{n} \sum_{k=1}^n (C_{k-1} + C_{n-k})$$

$$\rightarrow nC_n - (n-1)C_{n-1} = n(n+1) - (n-1)n + C_{n-1} + C_{n-1}$$

$$\rightarrow nC_n = (n+1)C_{n-1} + 2n$$

$$\rightarrow \text{Divide both sides by } n(n+1)$$



## 2.2. Quicksort

- Time complexity for sorting  $n$  elements
  - Abstract operation: *comparison*
  - Average case:

$$C_n = 2n \ln n = 1.38n \log_2 n = O(n \log_2 n)$$

Partition split can happen in each positions ( $1 \leq k \leq n$ ) with the same probability  $1/n$ .

$$C_0 = C_1 = 0$$

$$C_n = \frac{1}{n} \sum_{k=1}^n (C_{k-1} + C_{n-k} + (n+1)) \text{ when } n > 1$$

$$\rightarrow nC_n = (n+1)C_{n-1} + 2n$$

$$\rightarrow \frac{C_n}{n+1} = \frac{C_{n-1}}{n} + \frac{2}{n+1} = \frac{C_{n-2}}{n-1} + \frac{2}{n} + \frac{2}{n+1} = \dots = \frac{C_1}{2} + \frac{2}{3} + \dots + \frac{2}{n} + \frac{2}{n+1}$$

$$\rightarrow \frac{C_n}{n+1} = 2 \left( \frac{1}{3} + \dots + \frac{1}{n} + \frac{1}{n+1} \right) = 2 \left( 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} + \frac{1}{n+1} - \frac{3}{2} \right)$$



## 2.2. Quicksort

- Time complexity for sorting  $n$  elements
  - Abstract operation: *comparison*
  - Average case:

$$C_n = 2n \ln n = 1.38n \log_2 n = O(n \log_2 n)$$

Partition split can happen in each positions ( $1 \leq k \leq n$ ) with the same probability  $1/n$ .

$$C_0 = C_1 = 0$$

$$C_n = \frac{1}{n} \sum_{k=1}^n (C_{k-1} + C_{n-k} + (n+1)) \text{ when } n > 1$$

$$\Rightarrow nC_n = (n+1)C_{n-1} + 2n$$

$$\Rightarrow \frac{C_n}{n+1} \leq 2 \left( \ln n + 1 + \frac{1}{n+1} - \frac{3}{2} \right)$$

$$\Rightarrow C_n \approx 2n \ln n \approx 1.38n \log_2 n$$



## 2.2. Quicksort

- Time complexity for sorting  $n$  elements
  - Abstract operation: *comparison*
  - Average case:

$$C_n = 2n \ln n = 1.38n \log_2 n = O(n \log_2 n)$$

Partition split can happen in each positions ( $1 \leq k \leq n$ ) with the same probability  $1/n$ .

$$C_0 = C_1 = 0$$

$$C_n = \frac{1}{n} \sum_{k=1}^n (C_{k-1} + C_{n-k} + (n+1)) \text{ when } n > 1$$

$$\Rightarrow nC_n = (n+1)C_{n-1} + 2n$$

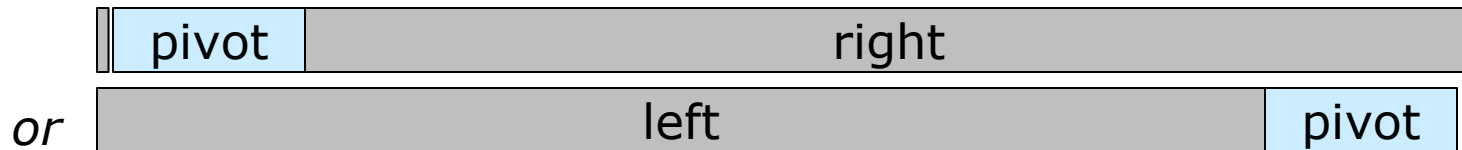
$$\Rightarrow \frac{C_n}{n+1} \leq 2 \left( \ln n + 1 + \frac{1}{n+1} - \frac{3}{2} \right)$$

$$\Rightarrow C_n \approx 2n \ln n \approx 1.38n \log_2 n$$

## 2.2. Quicksort

- Time complexity for sorting  $n$  elements
  - Abstract operation: *comparison*
  - Worst case:

$$C_n = (n^2 + 3n - 4)/2 = O(n^2)$$



Partitioning gives one empty and another of size  $n-1$ .

$$C_n = (n+1) + n + (n-1) + \dots + 3$$

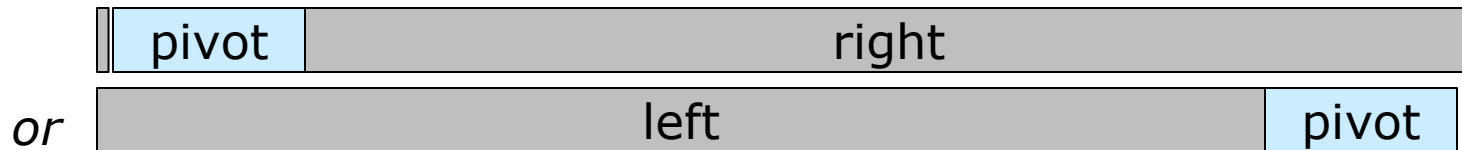
$$C_n = (n+1)(n+2)/2 - 3$$

$$C_n = (n^2 + 3n - 4)/2$$

## 2.2. Quicksort

- Time complexity for sorting  $n$  elements
  - Abstract operation: *comparison*
  - Worst case:

$$C_n = (n^2 + 3n - 4)/2 = O(n^2)$$



Partitioning gives one empty and another of size  $n-1$ .

$$C_n = (n+1) + n + (n-1) + \dots + 3$$

$$C_n = (n+1)(n+2)/2 - 3$$

$$C_n = (n^2 + 3n - 4)/2$$



## 2.2. Quicksort

---

### □ Your turn:

2.2.1. Trace by hand Quicksort with pivot = leftmost

(2.2.1.1) 23, 7, 92, 6, 12, 24, 40, 44, 20, 21

(2.2.1.2) 44, 30, 50, 22, 60, 55, 77, 55

(2.2.1.3) 2, 4, 6, 8, 10, 10

(2.2.1.4) 13, 11, 7, 7, 3, 2, 1



## 2.2. Quicksort

---

### □ Your turn:

2.2.2. Redesign Quicksort for pivot = rightmost. Is your Quicksort more efficient than original Quicksort?

2.2.3. Trace by hand your Quicksort with pivot = rightmost

(2.2.3.1) 23, 7, 92, 6, 12, 24, 40, 44, 20, 21

(2.2.3.2) 44, 30, 50, 22, 60, 55, 77, 55

(2.2.3.3) 2, 4, 6, 8, 10, 10

(2.2.3.4) 13, 11, 7, 7, 3, 2, 1



## 2.2. Quicksort

---

### □ Your turn:

2.2.4. Design and analyze an algorithm  $A$  to find a median in an unsorted array of  $n$  elements

2.2.5. Combine algorithm  $A$  with Quicksort to achieve a new algorithm to sort an array of  $n$  elements. Is it more efficient than original Quicksort? Why?





## 2.3. Mergesort

- To sort a given array of  $n$  elements, *divide it in half*, sort the two halves (recursively), and then *merge* the two halves together.
- **Merging**: given two sorted arrays  $a[1..M]$  and  $b[1..N]$ , we wish to merge them into the third sorted array  $c[1..M+N]$ .

$i := 1; j := 1;$

**for**  $k := 1$  **to**  $M+N$  **do**

**if**  $a[i] < b[j]$  **then**

**begin**  $c[k] := a[i]; i := i+1$  **end**

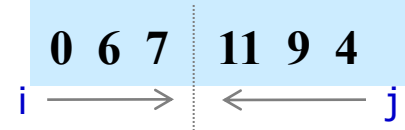
**else begin**  $c[k] := b[j]; j := j+1$  **end;**



## 2.3. Mergesort

```
procedure mergesort(left,right: integer);  
var i, j, k, m : integer;  
begin  
  if right-left>0 then  
    begin  
      m:=(right+left)/2; mergesort(left,m); mergesort(m+1,right);  
      for i := m downto left do b[i] := a[i];  
      for j :=m+1 to right do b[right+m+1-j] := a[j];  
      //i := left; j := right;  
      for k :=left to right do  
        if b[i] < b[j] then  
          begin a[k] := b[i] ; i := i+1 end  
          else begin a[k] := b[j]; j:= j-1 end;  
      end;  
    end;  
end;
```

**Merging on b:**



## 2.3. Mergesort

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|----|----|----|
| 8 | 8 | 9 | 2 | 1 | 0 | 0 | 7 | 6 | 11 | 4  | 9  |
| 8 | 8 | 9 | 2 | 1 | 0 |   |   |   |    |    |    |
| 8 | 8 | 9 |   |   |   |   |   |   |    |    |    |
| 8 | 8 |   |   |   |   |   |   |   |    |    |    |
| 8 |   |   |   |   |   |   |   |   |    |    |    |
|   | 8 |   |   |   |   |   |   |   |    |    |    |
| 8 | 8 |   |   |   |   |   |   |   |    |    |    |
|   |   | 9 |   |   |   |   |   |   |    |    |    |
| 8 | 8 | 9 |   |   |   |   |   |   |    |    |    |
|   |   |   | 2 | 1 |   |   |   |   |    |    |    |
|   |   |   | 2 |   |   |   |   |   |    |    |    |
|   |   |   |   | 1 |   |   |   |   |    |    |    |
|   |   |   | 1 | 2 |   |   |   |   |    |    |    |
|   |   |   |   |   | 0 |   |   |   |    |    |    |
|   |   |   | 0 | 1 | 2 |   |   |   |    |    |    |
| 0 | 1 | 2 | 8 | 8 | 9 |   |   |   |    |    |    |

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|----|----|----|
| 8 | 8 | 9 | 2 | 1 | 0 | 0 | 7 | 6 | 11 | 4  | 9  |
|   |   |   |   |   |   | 0 | 7 | 6 | 11 | 4  | 9  |
|   |   |   |   |   |   | 0 | 7 | 6 |    |    |    |
|   |   |   |   |   |   | 0 | 7 |   |    |    |    |
|   |   |   |   |   |   | 0 |   |   |    |    |    |
|   |   |   |   |   |   |   | 7 |   |    |    |    |
|   |   |   |   |   |   | 0 | 7 |   |    |    |    |
|   |   |   |   |   |   |   |   | 6 |    |    |    |
|   |   |   |   |   |   | 0 | 6 | 7 |    |    |    |
|   |   |   |   |   |   |   |   |   | 11 | 4  |    |
|   |   |   |   |   |   |   |   |   | 11 |    |    |
|   |   |   |   |   |   |   |   |   |    | 4  |    |
|   |   |   |   |   |   |   |   |   | 4  | 11 |    |
|   |   |   |   |   |   |   |   |   |    |    | 9  |
|   |   |   |   |   |   |   |   |   | 4  | 9  | 11 |
|   |   |   |   |   |   | 0 | 4 | 6 | 7  | 9  | 11 |
| 0 | 0 | 1 | 2 | 4 | 6 | 7 | 8 | 8 | 9  | 9  | 11 |



## 2.3. Mergesort

---

- Time complexity for sorting  $n$  elements
  - Abstract operation: *comparison*
  - All the cases:

$$C_1 = 0$$

$$C_n = 2C_{n/2} + n \quad \text{when } n \geq 2$$

$$C_n = n \log_2 n = O(n \log_2 n)$$

NOTE: Mergesort needs memory space of  $n$  elements for merging!



## 2.3. Mergesort

---

### □ Your turn:

#### ■ 2.3.1. Trace by hand Mergesort

(2.3.1.1) 23, 7, 92, 6, 12, 24, 40, 44, 20, 21

(2.3.1.2) 44, 30, 50, 22, 60, 55, 77, 55

(2.3.1.3) 2, 4, 6, 8, 10, 10

(2.3.1.4) 13, 11, 7, 7, 3, 2, 1



## 2.3. Mergesort

**Algorithm** *secret*( $a[1..M]$ ,  $b[1..N]$ ,  $c[1..M+N]$ )

//  $a$  and  $b$  are sorted arrays

**begin**

$i := 1$ ;  $j := 1$ ;  $k := 1$ ;

**while**  $i \leq M$  and  $j \leq N$  **do**

**begin**

**if**  $a[i] < b[j]$  **then**

**begin**

$c[k] := a[i]$  ;  $i := i + 1$ ;

**end**

**else**

**begin**

$c[k] := b[j]$ ;  $j := j + 1$

**end**;

$k := k + 1$ ;

**end**;

**if**  $i = M + 1$  **then**

**for**  $k1 := k + 1$  **to**  $M + N$  **do begin**  $c[k1] := b[j]$ ;  $j := j + 1$  **end**;

**if**  $j = N + 1$  **then**

**for**  $k1 := k + 1$  **to**  $M + N$  **do begin**  $c[k1] := a[i]$ ;  $i := i + 1$  **end**;

**end**;

### □ Your turn:

2.3.2. Given

*secret*, what does the algorithm perform? If *comparison* is an abstract operation, what is its complexity in the best, worst, and average cases?



## 2.4. External sorting

---

- ❑ Sorting the large files stored in secondary storage is called *external sorting*. External sorting is very important in database management systems.
- ❑ When estimating the computational time of the algorithms that work on files in hard disks, we must consider the number of times we read a block to main memory or write a block to secondary storage.
- ❑ Such operation is called *block access* or *disk access*.



## 2.4. External sorting

---

- The most commonly used technique for external sorting is the *external sort-merge* algorithm.
- This external sorting method has two stages.
  - *create runs*:  $b_r$  = number of blocks,  $M$  = buffer size
    - $\lceil b_r/M \rceil$  runs from any in-place in-memory sorting algorithm
  - *merge runs*: one block for output, each block of the rest for each input run
    - $\lceil \log_{(M-1)} \lceil b_r/M \rceil \rceil$  merging phases





## 2.4. External sorting

- *create runs*:  $\lceil b_r/M \rceil$  runs from in-place in-memory sorting

**i = 0;**

**repeat**

read M blocks of the file, or the rest of the file, whichever is smaller;

sort the in-memory part of the file;

write the sorted data to the run file  $R_i$ ;

$i := i+1$ ;

**until** the end of the file;

- *merge runs*: one block for output, each block of the rest for each input run with  $\lceil \log_{(M-1)} \lceil b_r/M \rceil \rceil$  merging phases



## 2.4. External sorting

- *create runs*:  $\lceil b_r/M \rceil$  runs from in-place in-memory sorting
- *merge runs*: one block for output, each block of the rest for each input run with  $\lceil \log_{(M-1)} \lceil b_r/M \rceil \rceil$  merging phases

**for** each merging phase

read one block of each of the  $N1$  files  $R_i$  into the buffer;  $//N1 \leq M-1$

**repeat**

choose the first tuple (in sort order) among all buffer pages;

write the tuple to the output and delete it from the buffer page;

**if** the buffer page of any run  $R_i$  is empty **and not** end-of-file( $R_i$ )

**then** read the next block of  $R_i$  into the buffer page;

**until** all buffer pages are empty;



## 2.4. External sorting

- File: 2, 3, 9, 1, 5, 6, 0, 9, 10, 16, 8, 11, 2, 15, 9, 4, 2, 0, 7, 4, 5, 3, 12, 13, 1, 0
  - 1 record/block, 26 records  $\Rightarrow b_r=26$  blocks
- $M = 4$  blocks
- Number of runs  $= \lceil b_r/M \rceil = 7$
- Number of merging phases  $= \lceil \log_{M-1} \lceil b_r/M \rceil \rceil = 2$

2,3,9,1, 5,6,0,9, 10,16,8,11, 2,15,9,4, 2,0,7,4, 5,3,12,13, 1,0

↓ *Generate runs*

1,2,3,9, 0,5,6,9, 8,10,11,16, 2,4,9,15, 0,2,4,7, 3,5,12,13, 0,1



## 2.4. External sorting

- File: 2, 3, 9, 1, 5, 6, 0, 9, 10, 16, 8, 11, 2, 15, 9, 4, 2, 0, 7, 4, 5, 3, 12, 13, 1, 0
  - 1 record/block, 26 records =>  $b_r=26$  blocks
- $M = 4$  blocks
- Number of runs =  $\lceil b_r/M \rceil = 7$
- Number of merging phases =  $\lceil \log_{M-1} \lceil b_r/M \rceil \rceil = 2$

1,2,3,9, 0,5,6,9, 8,10,11,16, 2,4,9,15, 0,2,4,7, 3,5,12,13, 0,1

↓ Merge in phase 1

0,1,2,3,5,6,8,9,9,10,11,16, 0,2,2,3,4,4,5,7,9,12,13,15, 0,1

↓ Merge in phase 2

0,0,0,1,1,2,2,2,3,3,4,4,5,5,6,7,8,9,9,9,10,11,12,13,15,16

## 2.4. External sorting

- Time complexity
  - Abstract operation: *block access*
  - All the cases:

$$2b_r + 2b_r \lceil \log_{M-1} \lceil b_r/M \rceil \rceil = 2b_r (\lceil \log_{M-1} \lceil b_r/M \rceil \rceil + 1)$$

run generation

run merging

$b_r$  = number of blocks,  
 $M$  = buffer size



## 2.4. External sorting

---

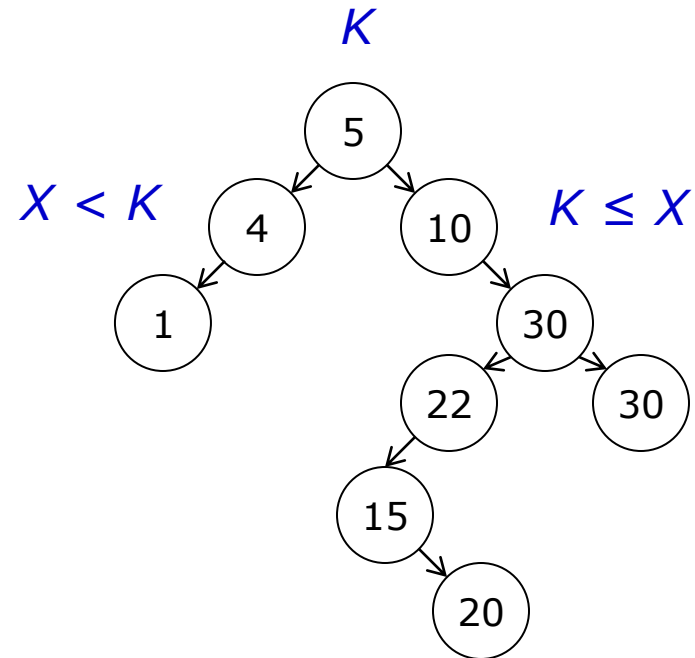
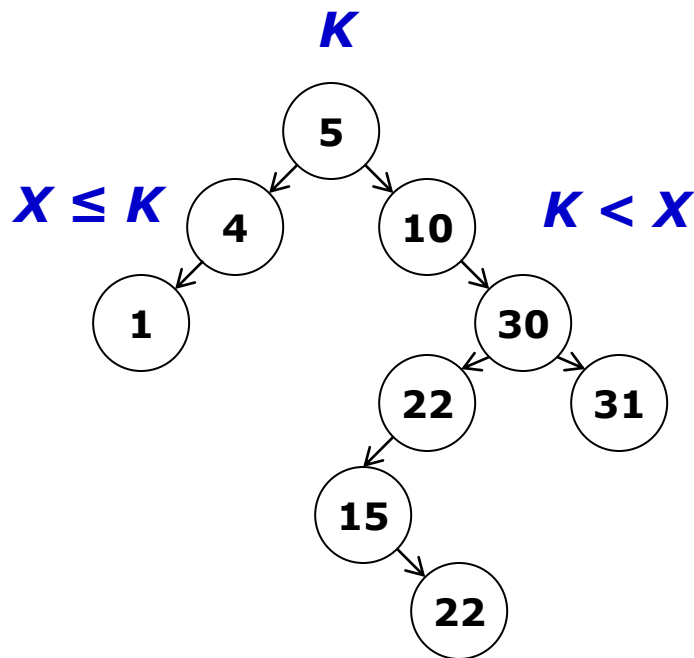
### □ Your turn:

2.4.1. Given the data file of 25 records with the following keys: 28, 3, 93, 10, 54, 65, 30, 90, 10, 69, 8, 22, 31, 5, 96, 40, 85, 9, 39, 13, 8, 77, 10, 2, 9.

Assume that one record fits in a block and memory buffer holds at most *three page frames (3 blocks)*. During the merge stage, two page frames are used for input and one for output. Trace by hand the *external sorting (external sort-merge)* for the above data file.

## 2.5. Binary search tree

- A binary search tree is a binary tree whose nodes contain elements of a set of orderable items, one element per node.





## 2.5. Binary search tree

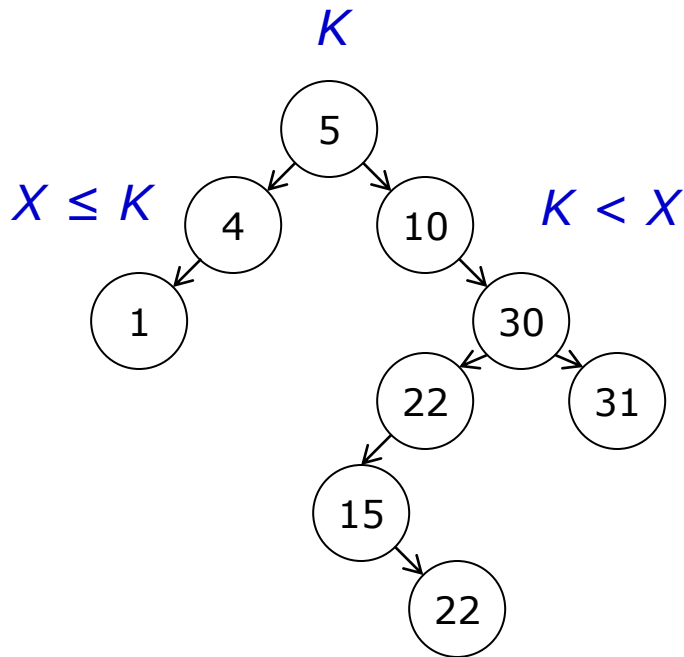
```
type link =  $\uparrow$  node;  
node = record key, info: integer;  
         l, r: link end;  
var t, head, z: link;
```

Insertion of a key  
into a binary search tree

```
procedure tree_insert (v: integer; x: link): link;  
var p: link;  
begin  
  repeat  
    p := x;  
    if v < x $\uparrow$ .key then x := x $\uparrow$ .l else x := x $\uparrow$ .r  
  until x = z;          /* z is a pseudo node for nil */  
  new(x); x $\uparrow$ .key := v;  
  x $\uparrow$ .l := z; x $\uparrow$ .r := z;    /* create a new node */  
  if v < p $\uparrow$ .key then p $\uparrow$ .l := x /* p denotes the parent of the new node */  
  else p $\uparrow$ .r := x;  
  tree_insert := x  
end
```

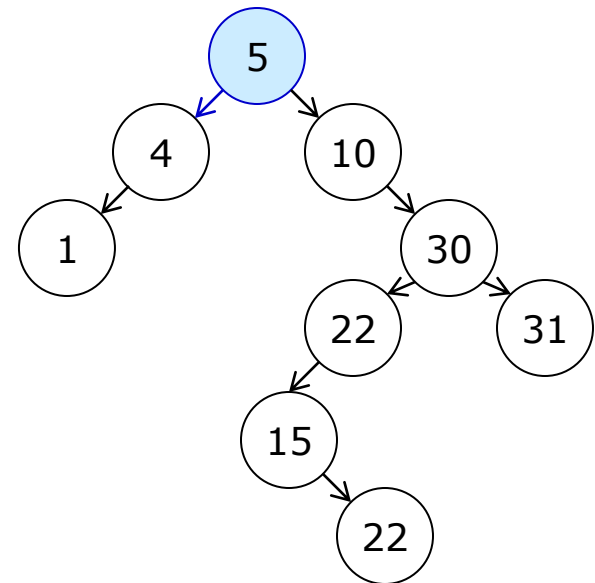


## 2.5. Binary search tree

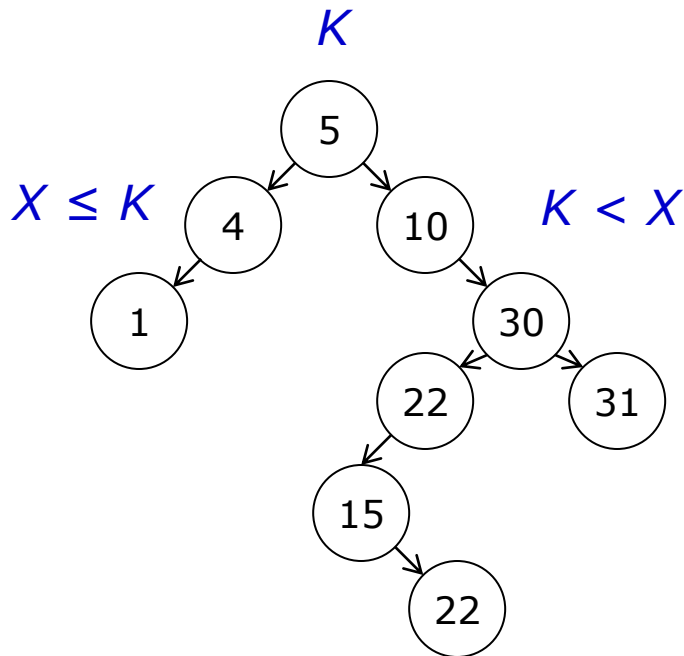


Insertion of a key, 3,  
into a binary search tree

$3 \leq 5$

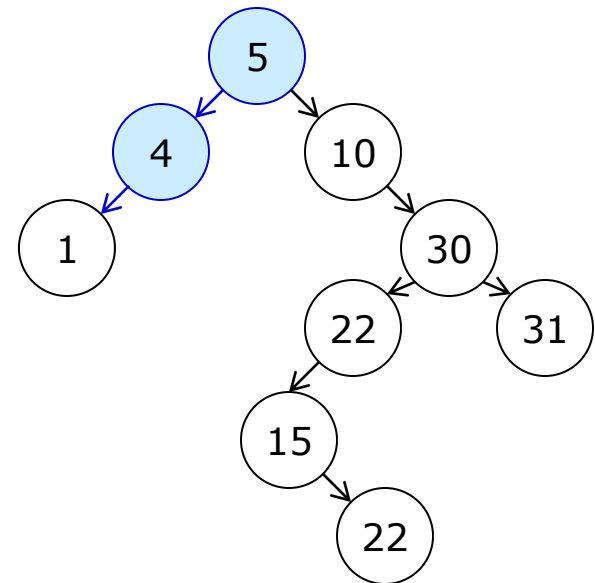



## 2.5. Binary search tree

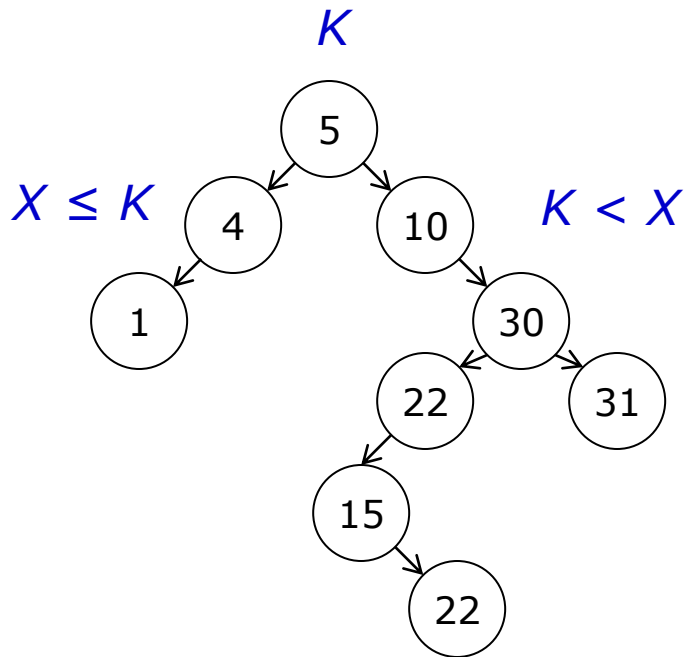


Insertion of a key, 3,  
into a binary search tree

$3 \leq 4$

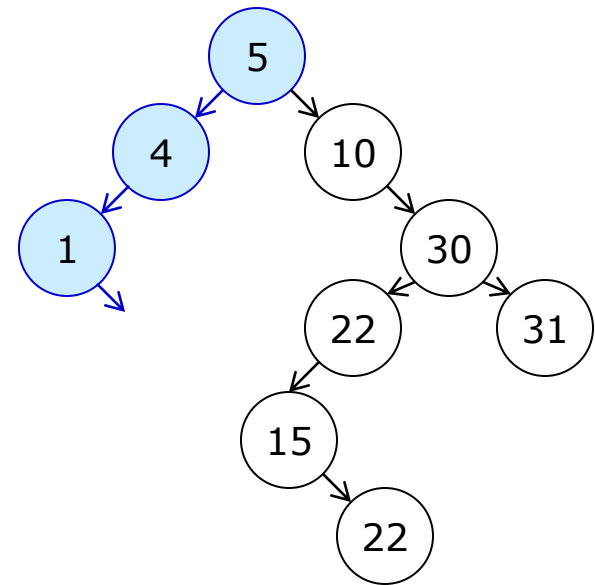



## 2.5. Binary search tree

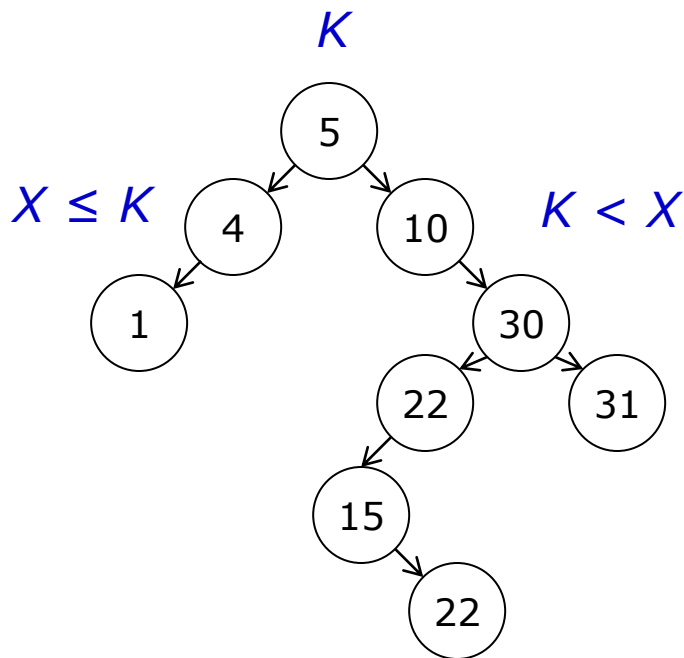


Insertion of a key, 3,  
into a binary search tree

$1 < 3$

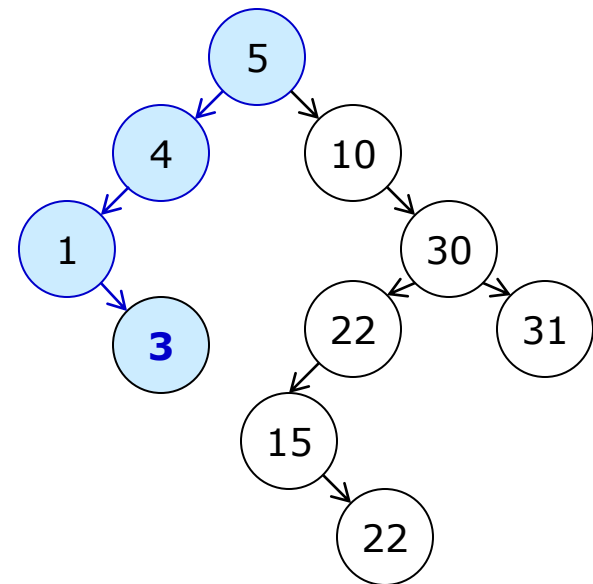



## 2.5. Binary search tree



Insertion of a key, 3, into a binary search tree

Insertion of 3





## 2.5. Binary search tree

### Searching in a binary search tree

*/\* search for the node with key v in the binary search tree x \*/*

```
function treesearch (v: integer, x: link): link;  
begin  
  while v <> x↑.key and x <> z do  
    begin  
      if v < x↑.key then x := x↑.l  
      else x := x↑.r  
    end;  
  treesearch := x  
end;
```



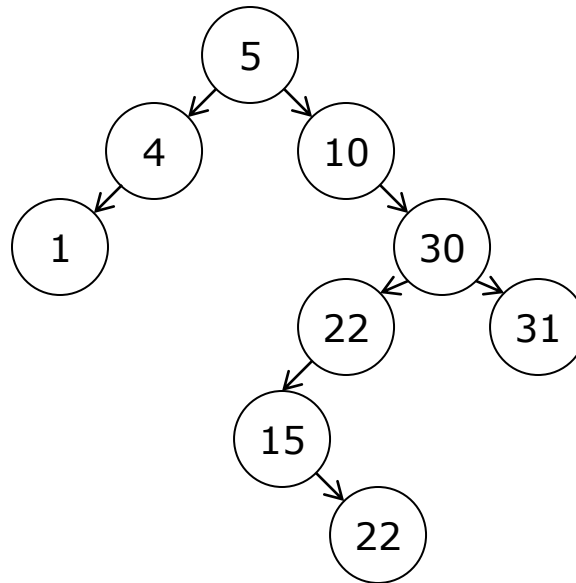
## 2.5. Binary search tree

---

- When we need to search for an element of a given value  $v$  in a binary search tree, we do it recursively in the following manner.
  - If the tree is empty, the search ends in failure. If the tree is not empty, we compare  $v$  with the tree's root  $K(r)$ . If they match, a desired element is found and the search can be stopped.
  - Otherwise, we continue with the search in the left subtree of the root if  $v < K(r)$  and in the right subtree if  $v > K(r)$ .

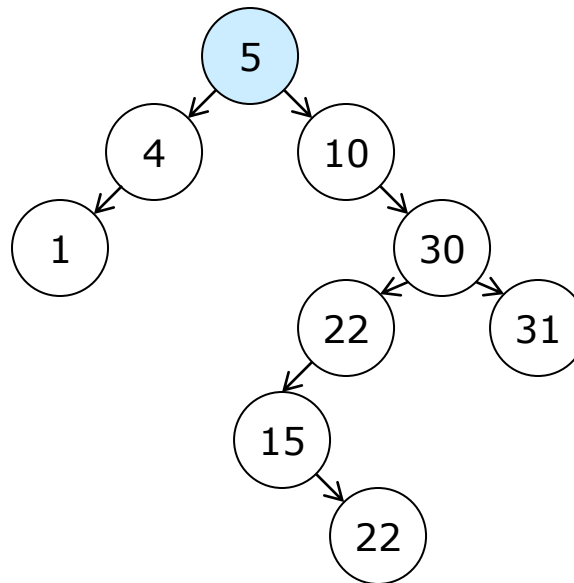
## 2.5. Binary search tree

□  $v = 15$ , is it in the following tree?



## 2.5. Binary search tree

□  $v = 15$ , is it in the following tree?

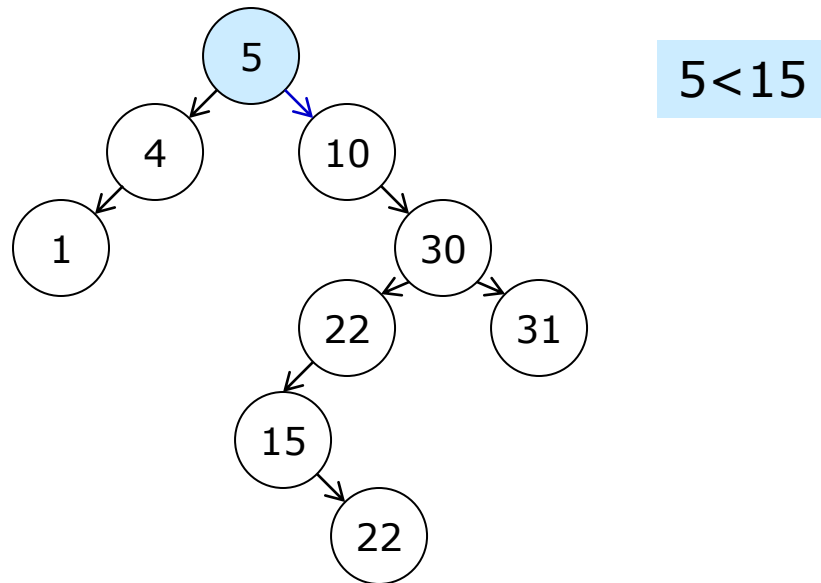


5=15 ?



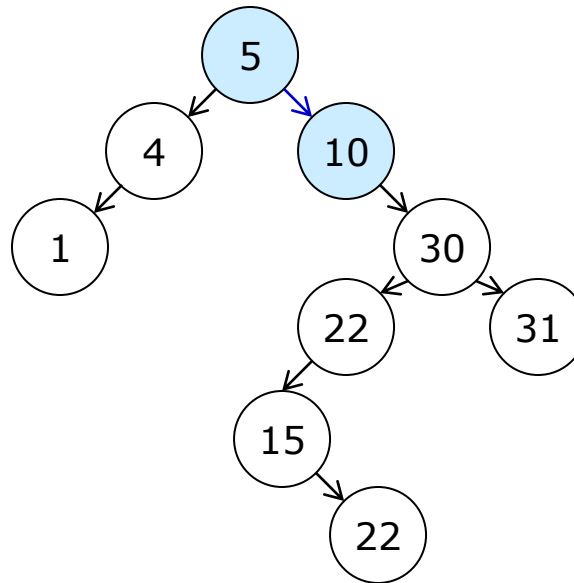
## 2.5. Binary search tree

□  $v = 15$ , is it in the following tree?



## 2.5. Binary search tree

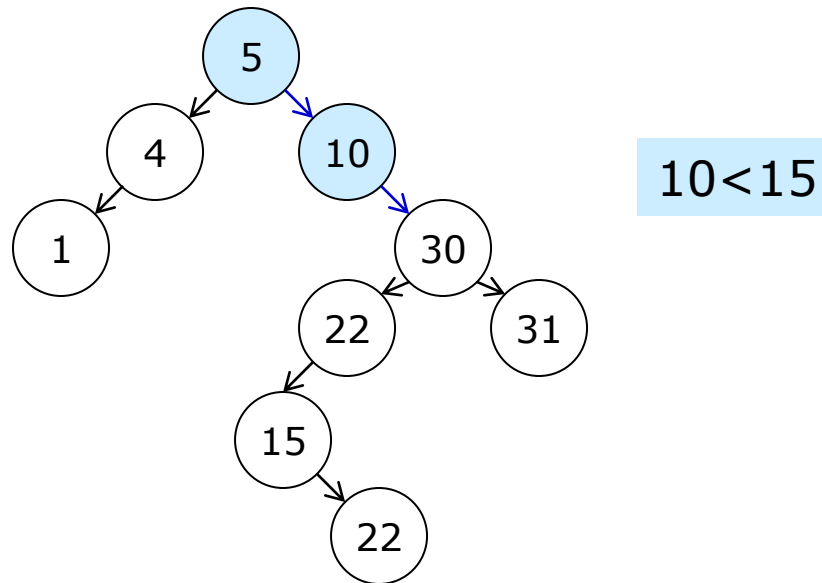
□  $v = 15$ , is it in the following tree?



10=15 ?

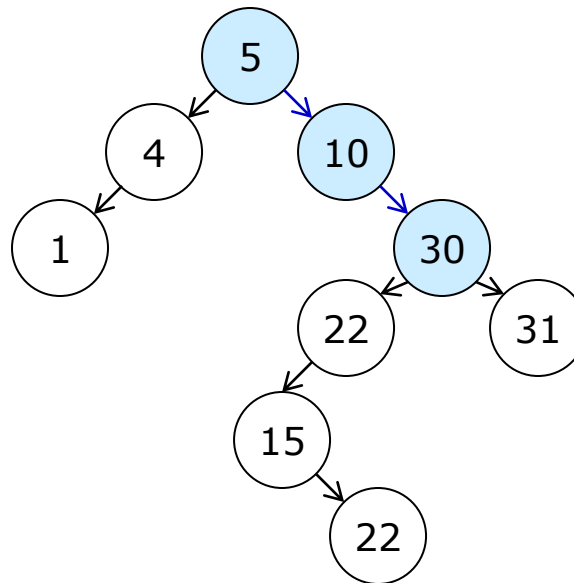
## 2.5. Binary search tree

□  $v = 15$ , is it in the following tree?



## 2.5. Binary search tree

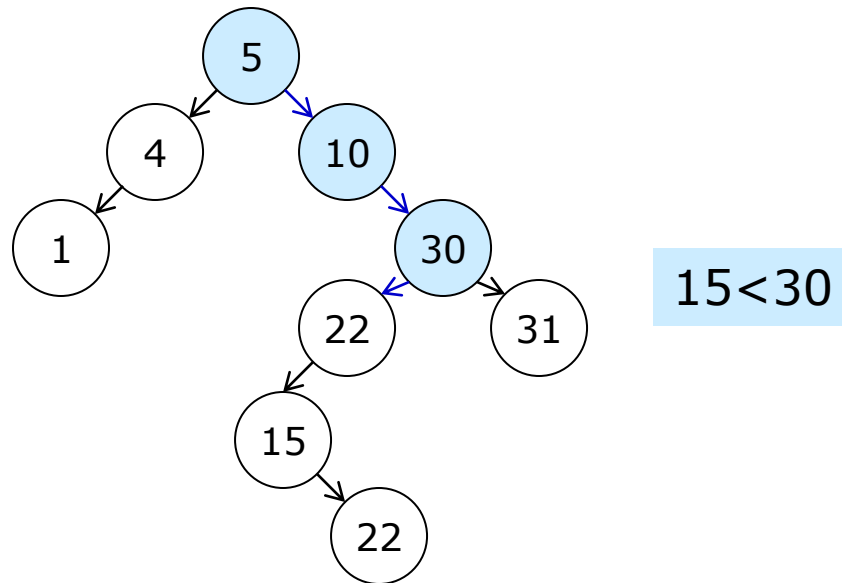
□  $v = 15$ , is it in the following tree?



30=15 ?

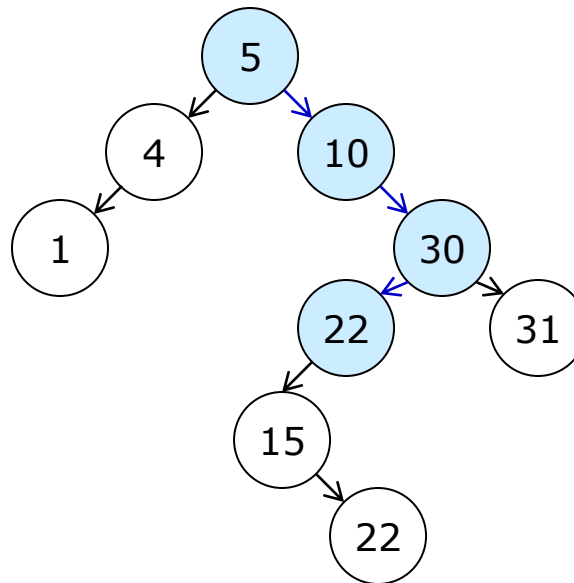
## 2.5. Binary search tree

□  $v = 15$ , is it in the following tree?



## 2.5. Binary search tree

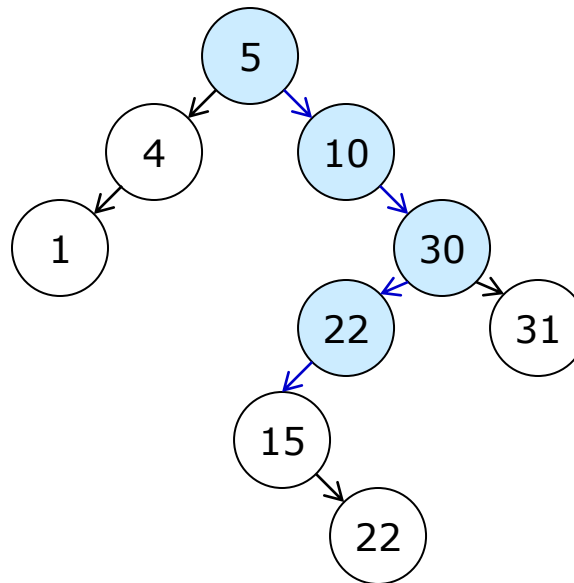
□  $v = 15$ , is it in the following tree?



22=15 ?

## 2.5. Binary search tree

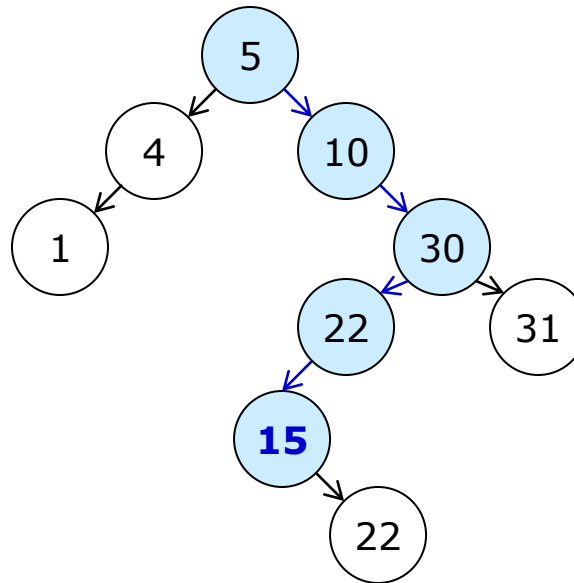
□  $v = 15$ , is it in the following tree?



15 < 22

## 2.5. Binary search tree

□  $v = 15$ , is it in the following tree?



15=15 ? **FOUND**





## 2.5. Binary search tree

---

- Time complexity for searching the tree of  $n$  keys for a given query key

- Abstract operation: *comparison*

- Best case:

$$C_n = 1 = O(1)$$

- Average case:

$$C_n = 2\ln n = 1.38\log_2 n = O(\log_2 n)$$

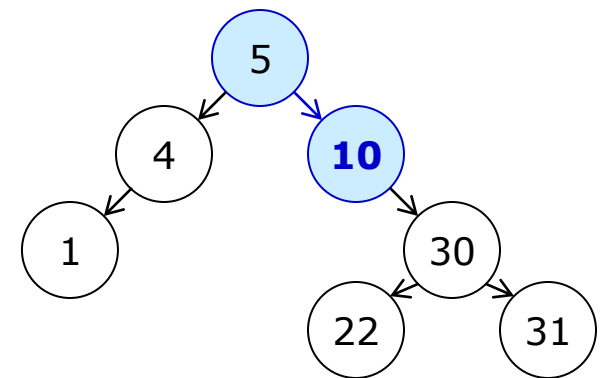
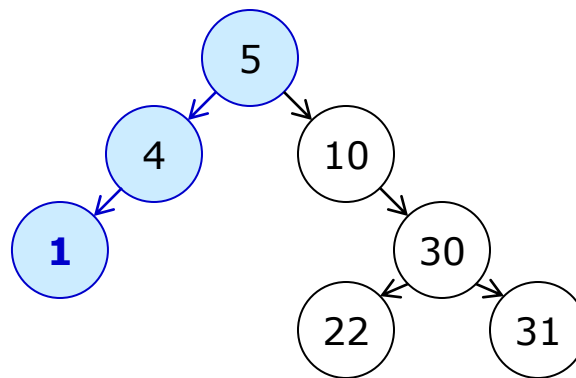
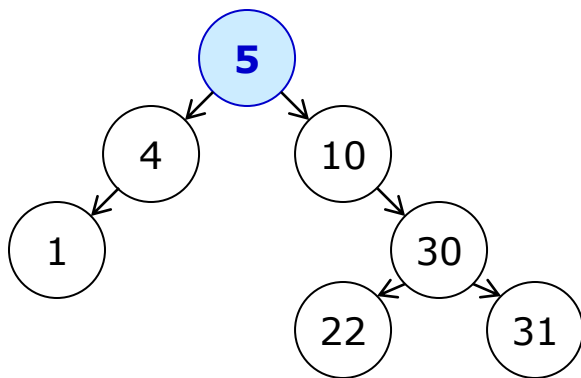
- Worst case:

$$C_n = n = O(n)$$

## 2.5. Binary search tree

- Time complexity for searching the tree of  $n$  keys for a given query key
  - Abstract operation: *comparison*
  - Average case:

$$C_n = 2\ln n = 1.38\log_2 n = O(\log_2 n)$$



Find 5:  $path(5) + 1$  cmps

Find 1:  $path(1) + 1$  cmps

Find 10:  $path(10) + 1$  cmps



## 2.5. Binary search tree

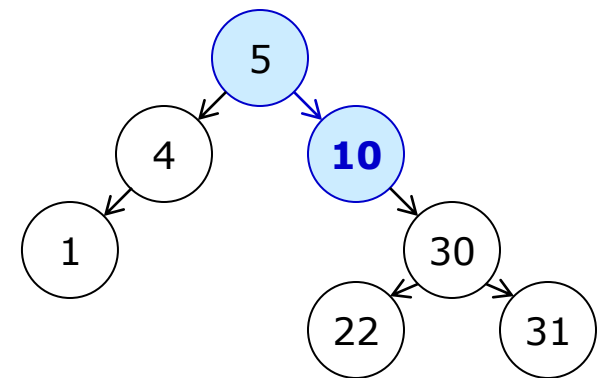
- Time complexity for searching the tree of  $n$  keys for a given query key
  - Abstract operation: *comparison*
  - Average case:

$$C_n = 2\ln n = 1.38\log_2 n = O(\log_2 n)$$

A query key can be found at any node  $k$  in the binary search tree. Each case has a probability of  $1/n$  and costs  $\text{path}(k) + 1$ .

$$C_n = \frac{1}{n} \sum_{k=1}^n (\text{path}(k) + 1) = \frac{1}{n} \left( n + \sum_{k=1}^n \text{path}(k) \right)$$

$\sum_{k=1}^n \text{path}(k)$  is the total sum of all the path lengths from the root to node  $k$  in the binary search tree of  $n$  nodes.



Find 10:  $\text{path}(10) + 1$  cmps 75



## 2.5. Binary search tree

- Time complexity for searching the tree of  $n$  keys for a given query key
  - Abstract operation: *comparison*
  - Average case:

$$C_n = 2\ln n = 1.38\log_2 n = O(\log_2 n)$$

A query key can be found at any node  $k$  in the binary search tree. Each case has a probability of  $1/n$  and costs  $\text{path}(k) + 1$ .

$$C_n = \frac{1}{n} \sum_{k=1}^n (\text{path}(k) + 1) = \frac{1}{n} \left( n + \sum_{k=1}^n \text{path}(k) \right) = 1 + \frac{D_n}{n}$$

$\sum_{k=1}^n \text{path}(k)$  is the total sum of all the path lengths from the root to node  $k$  in the binary search tree of  $n$  nodes.

$$D_n = \sum_{k=1}^n \text{path}(k) = \frac{1}{n} \sum_{i=1}^n (D_{i-1} + (i-1) + D_{n-i} + (n-i))$$



## 2.5. Binary search tree

- Time complexity for searching the tree of  $n$  keys for a given query key
  - Abstract operation: *comparison*
  - Average case:

$$C_n = 2\ln n = 1.38\log_2 n = O(\log_2 n)$$

A query key can be found at any node  $k$  in the binary search tree. Each case has a probability of  $1/n$  and costs  $\text{path}(k) + 1$ .

$$C_n = \frac{1}{n} \sum_{k=1}^n (\text{path}(k) + 1) = \frac{1}{n} \left( n + \sum_{k=1}^n \text{path}(k) \right) = 1 + \frac{D_n}{n}$$
$$D_n = \sum_{k=1}^n \text{path}(k) = \underbrace{\frac{1}{n} \sum_{i=1}^n}_{n \text{ cases of root } i} \underbrace{(D_{i-1} + (i-1))}_{\text{Left of root } i} + \underbrace{(D_{n-i} + (n-i))}_{\text{Right of root } i}$$

$n$  cases of root  $i$    Left of root  $i$    Right of root  $i$



## 2.5. Binary search tree

- Time complexity for searching the tree of  $n$  keys for a given query key
  - Abstract operation: *comparison*
  - Average case:

$$C_n = 2\ln n = 1.38\log_2 n = O(\log_2 n)$$

A query key can be found at any node  $k$  in the binary search tree. Each case has a probability of  $1/n$  and costs  $\text{path}(k) + 1$ .

$$C_n = \frac{1}{n} \sum_{k=1}^n (\text{path}(k) + 1) = \frac{1}{n} \left( n + \sum_{k=1}^n \text{path}(k) \right) = 1 + \frac{D_n}{n}$$

$$D_n = \sum_{k=1}^n \text{path}(k) = \frac{1}{n} \sum_{i=1}^n (D_{i-1} + (i-1) + D_{n-i} + (n-i))$$

$$D_n = \frac{1}{n} \sum_{i=1}^n (D_{i-1} + D_{n-i} + (n-1)) = \frac{2}{n} \sum_{i=1}^{n-1} D_i + n - 1 \text{ for } n > 1, D_1 = 0$$



## 2.5. Binary search tree

- Time complexity for searching the tree of  $n$  keys for a given query key
  - Abstract operation: *comparison*
  - Average case:

$$C_n = 2\ln n = 1.38\log_2 n = O(\log_2 n)$$

A query key can be found at any node  $k$  in the binary search tree. Each case has a probability of  $1/n$  and costs  $\text{path}(k) + 1$ .

$$C_n = \frac{1}{n} \sum_{k=1}^n (\text{path}(k) + 1) = \frac{1}{n} \left( n + \sum_{k=1}^n \text{path}(k) \right) = 1 + \frac{D_n}{n}$$

$$D_n = \frac{1}{n} \sum_{i=1}^n (D_{i-1} + D_{n-i} + (n-1)) = \frac{2}{n} \sum_{i=1}^{n-1} D_i + n - 1 \text{ for } n > 1, D_1 = 0$$

$$nD_n = 2 \sum_{i=1}^{n-1} D_i + n(n-1)$$

$$(n-1)D_{n-1} = 2 \sum_{i=1}^{n-2} D_i + (n-1)(n-2)$$



## 2.5. Binary search tree

- Time complexity for searching the tree of  $n$  keys for a given query key
  - Abstract operation: *comparison*
  - Average case:

$$C_n = 2\ln n = 1.38\log_2 n = O(\log_2 n)$$

A query key can be found at any node  $k$  in the binary search tree. Each case has a probability of  $1/n$  and costs  $\text{path}(k) + 1$ .

$$C_n = \frac{1}{n} \sum_{k=1}^n (\text{path}(k) + 1) = \frac{1}{n} \left( n + \sum_{k=1}^n \text{path}(k) \right) = 1 + \frac{D_n}{n}$$

$$D_n = \frac{1}{n} \sum_{i=1}^n (D_{i-1} + D_{n-i} + (n-1)) = \frac{2}{n} \sum_{i=1}^{n-1} D_i + n - 1 \text{ for } n > 1, D_1 = 0$$

$$nD_n - (n-1)D_{n-1} = 2D_{n-1} + 2(n-1) = 2D_{n-1} + 2n$$

$$nD_n = (n+1)D_{n-1} + 2n$$

$$D_n \approx 2n \ln n$$





## 2.5. Binary search tree

- Time complexity for searching the tree of  $n$  keys for a given query key
  - Abstract operation: *comparison*
  - Average case:

$$C_n = 2\ln n = 1.38\log_2 n = O(\log_2 n)$$

A query key can be found at any node  $k$  in the binary search tree. Each case has a probability of  $1/n$  and costs  $\text{path}(k) + 1$ .

$$C_n = \frac{1}{n} \sum_{k=1}^n (\text{path}(k) + 1) = \frac{1}{n} \left( n + \sum_{k=1}^n \text{path}(k) \right) = 1 + \frac{D_n}{n}$$

$$D_n = \frac{1}{n} \sum_{i=1}^n (D_{i-1} + D_{n-i} + (n-1)) = \frac{2}{n} \sum_{i=1}^{n-1} D_i + n - 1 \text{ for } n > 1, D_1 = 0$$

$$D_n \approx 2n\ln n$$

$$C_n \approx 1 + 2\ln n = 2\ln n = 1.38\log_2 n = O(\log_2 n)$$



## 2.5. Binary search tree

- Time complexity for searching the tree of  $n$  keys for a given query key
  - Abstract operation: *comparison*
  - Average case:

$$C_n = 2\ln n = 1.38\log_2 n = O(\log_2 n)$$

A query key can be found at any node  $k$  in the binary search tree. Each case has a probability of  $1/n$  and costs  $\text{path}(k) + 1$ .

$$C_n = \frac{1}{n} \sum_{k=1}^n (\text{path}(k) + 1) = \frac{1}{n} \left( n + \sum_{k=1}^n \text{path}(k) \right) = 1 + \frac{D_n}{n}$$

$$D_n = \frac{1}{n} \sum_{i=1}^n (D_{i-1} + D_{n-i} + (n-1)) = \frac{2}{n} \sum_{i=1}^{n-1} D_i + n - 1 \text{ for } n > 1, D_1 = 0$$

$$D_n \approx 2n\ln n$$

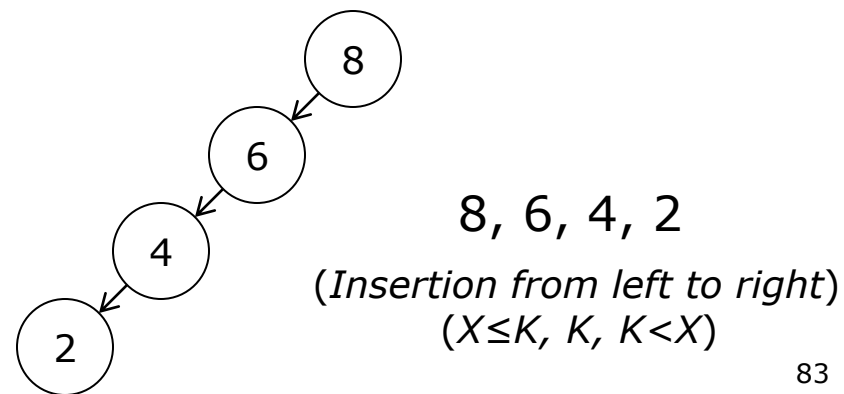
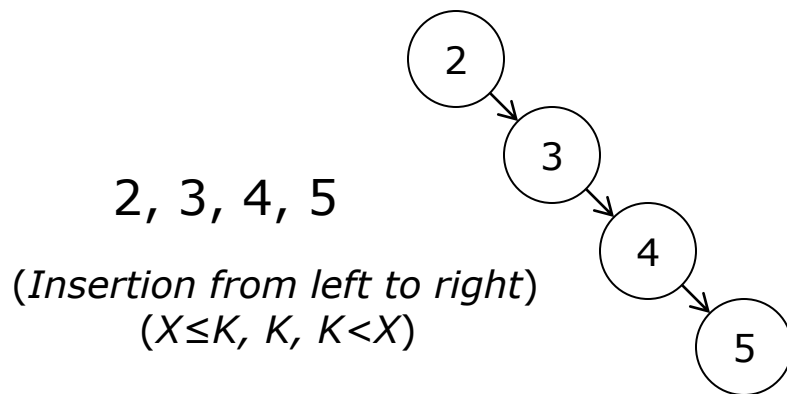
$$C_n \approx 1 + 2\ln n = 2\ln n = 1.38\log_2 n = O(\log_2 n)$$

## 2.5. Binary search tree

- Time complexity for searching the tree of  $n$  keys for a given query key
  - Abstract operation: *comparison*
  - Worst case:

$$C_n = n = O(n)$$

A binary search tree is deformed as a linked list.





## 2.5. Binary search tree

---

### □ Your turn:

2.5.1. Draw a *binary search tree* by inserting into an initially empty tree the following records from the left to right. Which one results in a binary search tree deformed as a linked list?

(2.5.1.1) 66, 33, 40, 22, 55, 88, 60, 11, 80, 20, 50, 44, 77, 30

(2.5.1.2) 2, 4, 6, 8, 10, 10

(2.5.1.3) 13, 11, 7, 7, 3, 2, 1

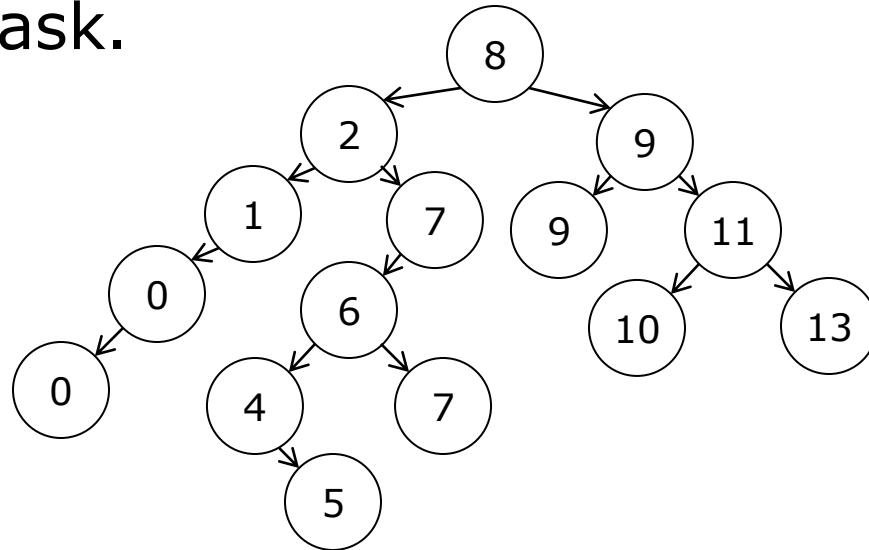
(2.5.1.4) 13, 8, 9, 2, 1, 0, 0, 7, 6, 11, 4, 9, 5

## 2.5. Binary search tree

### □ Your turn:

2.5.2. Given a binary search tree (BST), describe the paths to find *key 0* and *key 13* in this BST.

2.5.3. How is this BST updated if *key 8* is changed to *key 12*? Design and analyze an algorithm to perform this task.



# Algorithms Analysis and Design (C03031)

- Chapter 2. Divide-and-conquer
  - 2.1. Divide-and-conquer strategy
  - 2.2. Quicksort
  - 2.3. Mergesort
  - 2.4. External sorting
  - 2.5. Binary search tree

